



Broom AirLine

Documentacion Tecnica del Backend

C# + ASP.NET | UNIS 2026

Generado el 9 de abril de 2026



Indice de contenidos

GENERAL

- Inicio

API

- Controllers
- Services
- Repositories

ESTRUCTURA

- Models
- DTOs
- Helpers

INFRAESTRUCTURA

- Data
- Program

GENERAL

Inicio

Backend C# ASP.NET para la aerolinea Broom AirLine. Proyecto final UNIS 2026.

Tecnologias

- C# / ASP.NET Core
- Oracle Database
- Arquitectura por capas: Controllers, Services, Repositories

Estructura

- Controllers - Endpoints REST
- Services - Logica de negocio
- Repositories - Acceso a datos
- Models - Entidades
- DTOs - Objetos de transferencia
- Helpers - Utilidades
- Data - Conexion a base de datos
- Program - Punto de entrada de la aplicacion

API

Controllers

AdminVuelosController

Controlador de administracion de vuelos. Expone endpoints REST para crear, cancelar y consultar vuelos desde el panel de administrador. Todos los endpoints requieren rol Administrador.

```
CODIGO  
public async Task<IActionResult> CrearVuelo([FromBody] CrearVueloAdminDTO dto)
```

Crea un nuevo vuelo con los datos provistos por el administrador. Traduce errores de base de datos a mensajes legibles para el usuario.

```
CODIGO  
public async Task<IActionResult> ObtenerHistorialVuelos()
```

Retorna el historial completo de vuelos registrados en el sistema, incluyendo vuelos pasados, activos y cancelados.

```
CODIGO  
public async Task<IActionResult> CancelarVuelo(int id)
```

Cancela un vuelo activo por su identificador. Si el vuelo ya esta cancelado o no existe, retorna un error 404.

```
CODIGO  
public async Task<IActionResult> AvionesOcupados()
```

Devuelve los identificadores de aviones que ya tienen vuelo asignado en la fecha y hora indicadas. Permite filtrar adicionalmente por aeropuerto de origen. Se usa en el formulario de creacion de vuelo para deshabilitar aviones no disponibles.

```
CODIGO  
public async Task<IActionResult> TripulantesOcupados()
```

Devuelve los identificadores de tripulantes que ya tienen vuelo asignado en la fecha y hora indicadas. Se usa en el formulario de creacion de vuelo para deshabilitar tripulantes no disponibles.

AeropuertosController

Controlador de aeropuertos. Expone endpoints REST para consultar, crear, actualizar y eliminar aeropuertos, así como gestionar sus imagenes. Los endpoints de lectura son publicos; los de escritura requieren rol Administrador.

CODIGO

```
public async Task<IActionResult> ObtenerAeropuertos()
```

Retorna la lista completa de aeropuertos registrados. Endpoint publico, utilizado en el buscador de vuelos para poblar los selectores de origen y destino.

CODIGO

```
public async Task<IActionResult> ObtenerPorId(int id)
```

Retorna los datos de un aeropuerto especifico por su identificador. Devuelve 404 si el aeropuerto no existe.

CODIGO

```
public async Task<IActionResult> ObtenerFechasDisponibles(
```

Retorna las fechas con vuelos disponibles para una ruta determinada, filtrando por cantidad de pasajeros y clase. Se usa en el calendario del buscador de vuelos.

CODIGO

```
public async Task<IActionResult> Crear([FromBody] CrearAeropuertoDTO crearAeropuertoDTO)
```

Crea un nuevo aeropuerto con los datos del DTO. Requiere rol Administrador. Retorna 409 si ya existe un aeropuerto con el mismo codigo IATA.

CODIGO

```
public async Task<IActionResult> Actualizar(int id, [FromBody] CrearAeropuertoDTO actualizarAeropuertoDto)
```

Actualiza los datos de un aeropuerto existente. Requiere rol Administrador. Retorna 404 si el aeropuerto no existe y 409 si hay conflicto con el codigo IATA.

CODIGO

```
public async Task<IActionResult> Eliminar(int id)
```

Elimina un aeropuerto por su identificador. Requiere rol Administrador. Retorna 404 si el aeropuerto no existe.

CODIGO

```
public async Task<IActionResult> SubirImagen(int id, [FromBody] SubirImagenDTO dto)
```

Sube o reemplaza la imagen de un aeropuerto enviada como cadena Base64. Requiere rol Administrador. Retorna 404 si el aeropuerto no existe.

CODIGO

```
public async Task<IActionResult> EliminarImagen(int id)
```

Elimina la imagen asociada a un aeropuerto. Requiere rol Administrador. Retorna 404 si el aeropuerto no existe.

AgenciaController

Controlador de agencias de viaje. Expone endpoints para que el administrador gestione agencias y para que usuarios Webservice consulten y registren su propia agencia.

CODIGO

```
public async Task<IActionResult> CrearAgencia([FromBody] CrearAgenciaDTO dto)
```

Crea una nueva agencia vinculada a un usuario Webservice. Requiere rol Administrador. Verifica que el usuario no tenga ya una agencia ni un hotel aliado asignados.

CODIGO

```
public async Task<IActionResult> ObtenerTodasAdmin()
```

Retorna la lista completa de agencias registradas en el sistema. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ObtenerWebserviceDisponibles()
```

Retorna los usuarios con rol Webservice que no tienen ninguna entidad asignada (ni agencia ni hotel aliado). Se usa para poblar los selectores de asignacion tanto en el panel de agencias como en el de hoteles.

CODIGO

```
public async Task<IActionResult> AsignarUsuario(int id, [FromBody] AsignarUsuarioAgenciaDTO dto)
```

Asigna un usuario Webservice a una agencia existente. Requiere rol Administrador. El usuario no puede tener ya una agencia ni un hotel aliado registrado.

CODIGO

```
public async Task<IActionResult> ActualizarDescuento(int id, [FromBody] ActualizarDescuentoDTO dto)
```

Actualiza el porcentaje de descuento aplicado a las reservaciones de una agencia. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ActualizarEstado(int id, [FromBody] ActualizarEstadoAgenciaDTO dto)
```

Actualiza el estado de una agencia (activa, suspendida, etc.). Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ActualizarUrl(int id, [FromBody] ActualizarUrlAgenciaDTO dto)
```

Actualiza la URL publica de una agencia desde el panel de administracion. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ObtenerMiAgencia()
```

Retorna los datos de la agencia asociada al usuario Webservice autenticado. Si el usuario no tiene agencia registrada, retorna `tieneAgencia = false`. Solo accesible para usuarios con rol Webservice (`rolld = 3`).

CODIGO

```
public async Task<IActionResult> CrearMiAgencia([FromBody] CrearAgenciaWebserviceDTO dto)
```

Permite a un usuario Webservice autenticado crear su propia agencia por primera vez. Solo accesible para usuarios con rol Webservice (`rolld = 3`).

AsientoAgenciaController

Controlador de gestion de asientos para agencias de viaje. Permite a una agencia autenticada consultar y cambiar los asientos de los boletos dentro de sus reservaciones. Todos los endpoints requieren autenticacion de agencia mediante `AgenciaAuthMiddleware`.

CODIGO

```
public async Task<IActionResult> ObtenerAsientosPorReservacion(int reservacionId)
```

Retorna los asientos asignados a cada boleto de una reservacion especifica de la agencia. Verifica que la reservacion pertenezca a la agencia autenticada antes de retornar datos.

CODIGO

```
public async Task<IActionResult> CambiarAsiento(int boletoId, [FromBody] CambiarAsientoAgenciaRequestDTO dto)
```

Cambia el asiento asignado a un boleto especifico de una reservacion de la agencia. Verifica que el boleto pertenezca a la agencia autenticada antes de aplicar el cambio.

AsientoController

Controlador de gestion de asientos para usuarios autenticados. Permite al usuario consultar la disponibilidad de asientos en un vuelo y cambiar el asiento de su boleto. Todos los endpoints requieren sesion activa.

CODIGO

```
public async Task<IActionResult> ObtenerAsientosVuelo(int vueloId)
```

Devuelve la lista de asientos del vuelo indicado, marcando cuales estan ocupados y cual es el asiento actualmente asignado al usuario autenticado. Se usa para renderizar el mapa de asientos en el flujo de reservacion.

CODIGO

```
public async Task<IActionResult> CambiarAsiento(int boletoId, [FromBody] CambiarAsientoRequestDTO dto)
```

Cambia el asiento asignado al boleto indicado. Verifica que el boleto pertenezca al usuario autenticado y que el nuevo asiento este disponible antes de aplicar el cambio.

AuthController

Controlador de autenticacion. Gestiona el inicio de sesion, la consulta de sesion activa y el cierre de sesion mediante cookies cifradas de ASP.NET Core.

```
CODIGO
public async Task<IActionResult> Login(LoginRequestDto request)
```

```
CODIGO
public IActionResult ObtenerSesion()
```

Retorna los datos del usuario autenticado extraidos de la cookie de sesion activa, incluyendo id, nombre, correo, id de rol y nombre de rol. Requiere sesion activa.

```
CODIGO
public async Task<IActionResult> Logout()
```

Cierra la sesion del usuario eliminando la cookie de autenticacion del navegador. Requiere sesion activa.

AvionesController

Controlador de aviones. Expone endpoints REST para consultar, crear, actualizar, eliminar aviones y gestionar sus imagenes. Los endpoints de lectura son publicos; los de escritura requieren rol Administrador.

```
CODIGO
public async Task<ActionResult<List<AvionDTO>>> ObtenerTodos()
```

Retorna la lista completa de aviones registrados. Endpoint publico, utilizado para poblar selectores en el formulario de creacion de vuelos del panel de admin.

```
CODIGO
public async Task<ActionResult<AvionDTO>> ObtenerPorId(int id)
```

Retorna los datos de un avion especifico por su identificador. Devuelve 404 si el avion no existe.

```
CODIGO
public async Task<ActionResult<AvionDTO>> Crear([FromBody] CrearAvionDTO crearAvionDto)
```

Crea un nuevo avion con los datos del DTO. Requiere rol Administrador.

CODIGO

```
public async Task<ActionResult> Actualizar(int id, [FromBody] CrearAvionDTO actualizarAvionDto)
```

Actualiza los datos de un avion existente. Requiere rol Administrador. Retorna 404 si el avion no existe.

CODIGO

```
public async Task<ActionResult> Eliminar(int id)
```

Elimina un avion por su identificador. Requiere rol Administrador. Retorna 404 si el avion no existe.

CODIGO

```
public async Task<ActionResult> SubirImagen(int id, [FromBody] SubirImagenDTO dto)
```

Sube o reemplaza la imagen de un avion enviada como cadena Base64. Requiere rol Administrador. Retorna 404 si el avion no existe.

CODIGO

```
public async Task<ActionResult> EliminarImagen(int id)
```

Elimina la imagen asociada a un avion. Requiere rol Administrador. Retorna 404 si el avion no existe.

Busquedacontroller

Controlador de busquedas temporales de vuelos. Permite guardar los parametros de una busqueda en memoria y recuperarlos por identificador, facilitando la navegacion entre pasos del flujo de reservacion sin depender de la URL ni del estado del frontend.

CODIGO

```
public IActionResult GuardarBusqueda([FromBody] GuardarBusquedaDto dto)
```

Guarda los parametros de una busqueda de vuelos (origen, destino, fechas, pasajeros) en memoria y retorna un identificador unico para recuperarlos posteriormente.

CODIGO

```
public IActionResult ObtenerBusqueda(string id)
```

Recupera los parametros de una busqueda previamente guardada por su identificador. Retorna 404 si la busqueda no existe o ha expirado.

ComentariosController

Controlador de comentarios y respuestas sobre rutas. Permite a los usuarios autenticados publicar comentarios en rutas, responder comentarios existentes y consultar los comentarios con su estado de voto. Las agencias tambien pueden consultar comentarios de rutas.

CODIGO

```
public async Task<IActionResult> CrearComentarioRuta([FromBody] CrearComentarioRutaDTO dto)
```

Publica un nuevo comentario sobre una ruta especifica. Requiere sesion activa. El comentario queda vinculado al usuario autenticado y a la ruta indicada en el DTO.

CODIGO

```
public async Task<IActionResult> CrearRespuesta([FromBody] CrearRespuestaDTO dto)
```

Publica una respuesta a un comentario existente. Requiere sesion activa. La respuesta queda vinculada al comentario padre indicado en el DTO.

CODIGO

```
public async Task<IActionResult> ObtenerTodosConVoto()
```

Retorna todos los comentarios del sistema incluyendo el estado de voto del usuario autenticado en cada comentario. Requiere sesion activa.

CODIGO

```
public async Task<IActionResult> ObtenerMisComentarios()
```

Retorna los comentarios publicados por el usuario autenticado. Requiere sesion activa.

CODIGO

```
public async Task<IActionResult> ObtenerComentariosRutaConVoto(int rutaId)
```

Retorna los comentarios de una ruta especifica incluyendo el estado de voto del usuario autenticado en cada comentario. Requiere sesion activa.

CODIGO

```
public async Task<IActionResult> ObtenerComentariosPorRuta(int rutaId)
```

Retorna los comentarios de una ruta especifica. Endpoint publico, no requiere autenticacion.

CODIGO

```
public async Task<IActionResult> ObtenerComentariosRutaAgencia(int rutaId)
```

Retorna los comentarios de una ruta para consumo de agencias. Requiere autenticacion de agencia mediante AgenciaAuthMiddleware.

ConfirmarReservacionAgenciaController

Controlador para confirmar reservaciones de agencias de viaje. Expone el endpoint de confirmacion que finaliza el proceso de compra de una reservacion pendiente creada por una agencia autenticada.

```
CODIGO  
public async Task<IActionResult> ConfirmarReservacion()
```

Confirma y finaliza el pago de una reservacion existente de la agencia. Verifica que la reservacion pertenezca a la agencia autenticada antes de procesar la confirmacion.

EmailController

Controlador de correos electronicos. Gestiona el envio de comprobantes de reservacion, mensajes de contacto desde el formulario publico y suscripciones al boletin informativo.

```
CODIGO  
public async Task<IActionResult> EnviarCorreoReservacion(int id)
```

Envia al correo del usuario el comprobante de una reservacion especifica. Solo accesible para roles Administrador (1) y Cliente (2). Verifica que la reservacion pertenezca al usuario autenticado antes de enviar el correo.

```
CODIGO  
public async Task<IActionResult> Contacto([FromBody] ContactoDTO dto)
```

Recibe un mensaje del formulario de contacto publico y lo reenvía al correo del administrador. Requiere nombre, correo y mensaje como campos obligatorios.

```
CODIGO  
public async Task<IActionResult> Newsletter([FromBody] NewsletterDTO dto)
```

Registra una suscripcion al boletin informativo y notifica al administrador por correo. Endpoint publico, valida que el correo tenga formato valido antes de procesar.

FacturaController

Controlador de facturacion y compra de reservaciones. Expone el endpoint que procesa el pago de una reservacion pendiente, genera la factura y confirma los boletos del usuario.

```
CODIGO  
public async Task<IActionResult> ComprarReservacion(int id, [FromBody] ComprarReservacionDTO dto)
```

Procesa la compra de una reservacion pendiente validando el metodo de pago y generando la factura correspondiente. Requiere sesion activa. Solo el propietario de la reservacion puede realizar la compra.

HandshakeController

Controlador de autenticacion de agencias externas (handshake). Permite a una agencia externa autenticarse contra la API mediante un token de entrada y recibir un token de sesion para usar en las solicitudes posteriores a los endpoints de agencia.

CODIGO

```
public async Task<IActionResult> ProcesarHandshake([FromBody] HandshakeRequestDTO dto)
```

Recibe las credenciales de una agencia externa (URL y token de entrada), las valida contra la base de datos y retorna un token de sesion si la autenticacion es exitosa.

HandshakeHotelController

Controlador que inicia el proceso de handshake entre la aerolinea y un hotel aliado. Genera un token de entrada, lo envia al hotel y guarda el token de sesion resultante en la base de datos. Solo accesible por administradores autenticados.

CODIGO

```
public async Task<IActionResult> IniciarHandshake(int hotelId)
```

Inicia el handshake de autenticacion con el hotel aliado identificado por hotelId. Genera un token de entrada, lo envia al endpoint /api/aerolineas/handshake del hotel y guarda el token de sesion recibido en HotelAliado.TokenHASH. Retorna el token de sesion resultante si el proceso fue exitoso.

HealthController

Controlador de estado de la API. Expone un endpoint publico para verificar que el servicio esta en linea y obtener informacion basica del ambiente y la hora del servidor. Utilizado por herramientas de monitoreo y orquestacion de contenedores.

CODIGO

```
public IActionResult Get()
```

Retorna el estado actual de la API, el ambiente de ejecucion (Development, Production, etc.) y la hora UTC del servidor. Endpoint publico, no requiere autenticacion.

HotelAliadoController

Controlador de hoteles aliados. Expone el endpoint de busqueda dinamica de hoteles, los endpoints para que usuarios Webservice consulten y registren su propio hotel, y los endpoints de administracion completa para el panel de administrador.

CODIGO

```
public async Task<IActionResult> BuscarHoteles([FromBody] BusquedaHotelesDTO dto)
```

Busca hoteles disponibles en la ciudad destino del pasajero consultando la API de cada hotel aliado activo de forma dinamica.

CODIGO

```
public async Task<IActionResult> ObtenerMiHotel()
```

Retorna los datos del hotel aliado asociado al usuario Webservice autenticado. Si el usuario no tiene hotel registrado, retorna tieneHotel = false. Solo accesible para usuarios con rol Webservice (roleId = 3).

CODIGO

```
public async Task<IActionResult> CrearMiHotel([FromBody] CrearHotelWebserviceDTO dto)
```

Permite a un usuario Webservice autenticado registrar su propio hotel aliado. Un usuario Webservice solo puede tener un hotel o una agencia, nunca ambos. Solo accesible para usuarios con rol Webservice (roleId = 3).

CODIGO

```
public async Task<IActionResult> ObtenerTodosAdmin()
```

Retorna la lista completa de hoteles aliados con datos del usuario asignado. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> CrearHotelAdmin([FromBody] CrearHotelAdminDTO dto)
```

Crea un nuevo hotel aliado y lo vincula al usuario Webservice indicado. Verifica que el usuario no tenga ya ninguna otra entidad asignada. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ActualizarEstado(int id, [FromBody] ActualizarEstadoHotelDTO dto)
```

Actualiza el estado de un hotel aliado segun el catalogo EstadoAliado. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> AsignarUsuario(int id, [FromBody] AsignarUsuarioHotelDTO dto)
```

Asigna un usuario Webservice a un hotel aliado existente. El usuario no puede tener ya ninguna otra entidad asignada. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ActualizarUrls(int id, [FromBody] ActualizarUrlHotelDTO dto)
```

Actualiza la URL de la API y la URL publica para usuarios de un hotel aliado. Requiere rol Administrador.

MetricasController

Controlador de metricas y analiticos del sistema. Expone endpoints para que el administrador consulte resúmenes, gráficas de búsquedas por día, rutas más buscadas, búsquedas por tipo de canal, listados con filtros paginados y exportación de reportes por correo. Todos los endpoints requieren rol Administrador.

CODIGO

```
public async Task<IActionResult> ObtenerResumen()
```

Retorna un resumen de metricas clave del sistema (totales, conversiones, etc.) para el rango de fechas indicado. Si no se especifican fechas se usa el periodo completo.

CODIGO

```
public async Task<IActionResult> BúsquedasPorDia()
```

Retorna la cantidad de búsquedas realizadas por día dentro del rango de fechas indicado. Se usa para renderizar la gráfica de línea en el panel de analiticos del administrador.

CODIGO

```
public async Task<IActionResult> RutasMasBuscadas()
```

Retorna las rutas origen-destino más frecuentes en el periodo indicado, con opción de filtrar por tipo de canal (Web o REST). Se usa para la gráfica de barras del panel.

CODIGO

```
public async Task<IActionResult> BúsquedasPorTipo()
```

Retorna el desglose de búsquedas por tipo de canal (Web vs REST) en el periodo indicado. Se usa para la gráfica de dona del panel de analiticos.

CODIGO

```
public async Task<IActionResult> ObtenerListado([FromBody] MetricasFiltroDTO filtro)
```

Retorna un listado paginado de registros de búsqueda con los filtros especificados en el cuerpo de la solicitud (fechas, tipo de canal, usuario y tamaño de página).

CODIGO

```
public async Task<IActionResult> ExportarPorCorreo([FromBody] ExportarMetricasDTO dto)
```

Genera un reporte HTML con todos los registros de búsqueda según los filtros indicados y lo envía por correo electrónico a la dirección especificada. No aplica paginación al exportar, incluye hasta 9999 registros.

MisReservacionesController

Controlador de reservaciones del usuario autenticado. Permite consultar el listado y detalle de sus reservaciones, descargar o enviar comprobantes, obtener un resumen estadístico y cancelar reservaciones activas. Todos los endpoints requieren sesion activa.

```
CODIGO  
public async Task<IActionResult> ObtenerMisReservaciones()
```

Retorna el listado de todas las reservaciones del usuario autenticado, incluyendo estado, vuelos y monto total de cada una.

```
CODIGO  
public async Task<IActionResult> ObtenerDetalleReservacion(int reservacionId)
```

Retorna el detalle completo de una reservacion especifica del usuario autenticado, incluyendo boletos, pasajeros, vuelos y datos de facturacion.

```
CODIGO  
public async Task<IActionResult> DescargarComprobante(int reservacionId)
```

Genera y retorna el comprobante de una reservacion como HTML para que el usuario lo abra en una nueva pestana e imprima como PDF desde el navegador. No requiere la libreria nativa wkhtmltopdf.

```
CODIGO  
public async Task<IActionResult> ObtenerResumen()
```

Retorna un resumen estadistico de las reservaciones del usuario autenticado, como totales por estado, monto gastado y proximos vuelos.

```
CODIGO  
public async Task<IActionResult> CancelarReservacion(int reservacionId, [FromBody] CancelarReservacionDTO dto)
```

Cancela una reservacion activa del usuario autenticado. El motivo de cancelacion es opcional. Solo se pueden cancelar reservaciones pendientes o confirmadas.

```
CODIGO  
public async Task<IActionResult> EnviarComprobanteEmail(int reservacionId)
```

Envia el comprobante de una reservacion al correo electronico registrado del usuario. Genera el HTML del comprobante y lo adjunta al correo antes de enviarlo.

NacionalidadesController

Controlador de nacionalidades. Expone un endpoint publico para obtener el catalogo de nacionalidades disponibles, utilizado en el formulario de registro de usuarios y en el ingreso de datos de pasajeros durante la reservacion.

```
CODIGO  
public async Task<IActionResult> Get()
```

Retorna el listado completo de nacionalidades registradas en el sistema. Endpoint publico, no requiere autenticacion.

PdfController

Controlador de generacion de comprobantes en HTML para impresion como PDF. Genera el HTML del comprobante de una reservacion y lo retorna como contenido de texto para que el frontend lo abra en una nueva pestana e imprima desde el navegador.

```
CODIGO  
public async Task<IActionResult> ObtenerComprobante(int id)
```

Retorna el HTML formateado del comprobante de una reservacion para que el usuario lo imprima como PDF desde el navegador. Solo accesible para roles Administrador (1) y Cliente (2). Verifica que la reservacion pertenezca al usuario autenticado.

PerfilController

Controlador de perfil de usuario. Permite al usuario autenticado consultar sus datos, actualizar su numero de telefono y cambiar su contraseña. Aplica verificacion de propiedad para garantizar que cada usuario solo pueda modificar su propio perfil. Todos los endpoints requieren sesion activa.

```
CODIGO  
public async Task<IActionResult> ObtenerPerfil(int usuarioId)
```

Retorna los datos del perfil de un usuario especifico. Solo el propio usuario puede consultar su perfil; retorna 403 si el id de ruta no coincide con la sesion.

```
CODIGO  
public async Task<IActionResult> ActualizarTelefono()
```

Actualiza el numero de telefono del usuario especificado. Solo el propio usuario puede modificar su telefono; retorna 403 si el id de ruta no coincide con la sesion.

```
CODIGO  
public async Task<IActionResult> CambiarContraseña()
```

Cambia la contraseña del usuario especificado tras validar la contraseña actual. Solo el propio usuario puede cambiar su contraseña; retorna 403 si el id de ruta no coincide con la sesion activa.

ReservacionAgenciaController

Controlador para la creacion y gestion inicial de reservaciones por parte de agencias de viaje. Permite a una agencia autenticada crear reservaciones, agregar datos de pasajeros y expirar reservaciones pendientes. Todos los endpoints requieren autenticacion de agencia.

CODIGO

```
public async Task<IActionResult> CrearReservacion([FromBody] CrearReservacionDTO dto)
```

Crea una nueva reservacion en estado pendiente para la agencia autenticada. Retorna el detalle de la reservacion creada incluyendo los boletos generados.

CODIGO

```
public async Task<IActionResult> ExpirarReservacion(int id)
```

Marca una reservacion pendiente como expirada. Se usa cuando la agencia no completa el flujo de confirmacion dentro del tiempo permitido.

CODIGO

```
public async Task<IActionResult> AgregarPasajeros([FromBody] AgregarPasajerosDTO dto)
```

Guarda los datos de los pasajeros asociados a una reservacion de la agencia. Debe llamarse antes de confirmar la reservacion para registrar la informacion requerida de cada boleto.

ReservacionesAgenciaController

Controlador de gestion de reservaciones existentes para agencias de viaje. Permite a una agencia autenticada consultar el detalle de una reservacion, cancelarla y verificar si es posible cancelarla. Todos los endpoints requieren autenticacion de agencia.

CODIGO

```
public async Task<IActionResult> ObtenerDetalle(int reservacionId)
```

Retorna el detalle completo de una reservacion especifica de la agencia autenticada, incluyendo boletos, pasajeros, vuelos y estado actual.

CODIGO

```
public async Task<IActionResult> CancelarReservacion(int reservacionId, [FromBody] CancelarReservacionDTO dto)
```

Cancela una reservacion activa de la agencia autenticada. El motivo de cancelacion es opcional. Solo se pueden cancelar reservaciones que aun no hayan sido completadas.

CODIGO

```
public async Task<IActionResult> PuedeCancelar(int reservacionId)
```

Verifica si una reservacion de la agencia puede ser cancelada segun las reglas de negocio (estado actual, tiempo antes del vuelo, etc.). Retorna un objeto con el resultado de la validacion.

ReservacionesController

Controlador de reservaciones para usuarios del portal web. Expone los endpoints para crear una reservacion y agregar los datos de los pasajeros asociados. La creacion de reservacion requiere sesion activa; el alta de pasajeros tambien.

CODIGO

```
public async Task<IActionResult> CrearReservacion([FromBody] CrearReservacionDTO dto)
```

Crea una nueva reservacion en estado pendiente para el usuario autenticado. Asigna automaticamente los boletos y asientos segun el vuelo y la clase seleccionados.

CODIGO

```
public async Task<IActionResult> AgregarPasajeros(int id, [FromBody] List<DatosPasajeroDTO> pasajeros)
```

Registra o actualiza los datos de los pasajeros de una reservacion existente. Debe llamarse antes de confirmar la compra para asociar la informacion de cada pasajero al boleto correspondiente.

RutaAgenciaController

Controlador de rutas para agencias de viaje externas. Expone el listado de rutas disponibles a agencias autenticadas mediante AgenciaAuthMiddleware, para que puedan construir sus propios buscadores de vuelos.

CODIGO

```
public async Task<IActionResult> ObtenerRutas()
```

Retorna el listado completo de rutas con vuelos disponibles para consumo de agencias. Requiere autenticacion de agencia mediante token en la solicitud.

RutasController

Controlador de rutas de vuelo. Expone endpoints para que el administrador consulte, cree y actualice rutas origen-destino, asi como calcule el tiempo estimado de llegada teniendo en cuenta las zonas horarias de los aeropuertos involucrados. Todos los endpoints requieren rol Administrador.

CODIGO

```
public async Task<IActionResult> ObtenerTodas()
```

Retorna el listado completo de rutas registradas en el sistema. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ActualizarDuracion(int id, [FromBody] EditarDuracionRutaDTO dto)
```

Actualiza la duracion estimada de vuelo de una ruta existente. Requiere rol Administrador. Retorna 404 si la ruta no existe y 400 si la duracion proporcionada no es valida.

CODIGO

```
public async Task<IActionResult> CalcularLlegada([FromBody] CalculoLlegadaRequestDTO request)
```

Calcula la fecha y hora de llegada estimada a partir del aeropuerto origen, destino, fecha y hora de salida. Considera las zonas horarias de ambos aeropuertos. Retorna null si faltan datos en lugar de retornar error, para no bloquear el formulario del admin. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> CrearRuta([FromBody] CrearRutaDTO dto)
```

Crea una nueva ruta entre dos aeropuertos con la duracion estimada indicada. Si la ruta ya existe retorna el mensaje correspondiente. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> ExisteRuta()
```

Verifica si ya existe una ruta entre dos aeropuertos. Se usa en el formulario de creacion de rutas para validar en tiempo real. Requiere rol Administrador.

TokenHotelController

*Controlador que expone el endpoint para solicitar un token de alianza a un hotel aliado específico.
Requiere sesion activa del usuario.*

CODIGO

```
public async Task<IActionResult> SolicitarToken(int aliadoId, [FromBody] TokenHotelRequestDTO dto)
```

Solicita un token de alianza al hotel identificado por aliadoId. El hotel genera un token de un solo uso valido por 15 minutos y retorna la URL lista para redirigir al usuario.

TripulacionController

Controlador de tripulacion. Expone endpoints REST para consultar, crear, actualizar y eliminar tripulantes, asi como gestionar sus imagenes y obtener el catalogo de roles. Los endpoints de lectura son publicos; los de escritura requieren rol Administrador.

CODIGO

```
public async Task<IActionResult> ObtenerTodos()
```

Retorna la lista completa de tripulantes registrados. Endpoint publico, utilizado para poblar selectores en el formulario de creacion de vuelos del panel de admin.

CODIGO

```
public async Task<IActionResult> ObtenerPorId(int id)
```

Retorna los datos de un tripulante especifico por su identificador. Devuelve 404 si el tripulante no existe.

CODIGO

```
public async Task<IActionResult> ObtenerRoles()
```

Retorna el catalogo de roles de tripulacion disponibles (piloto, copiloto, auxiliar de vuelo, etc.). Endpoint publico.

CODIGO

```
public async Task<IActionResult> Crear([FromBody] CrearTripulanteDTO crearTripulanteDTO)
```

Crea un nuevo tripulante con los datos del DTO. Requiere rol Administrador.

CODIGO

```
public async Task<IActionResult> Actualizar(int id, [FromBody] CrearTripulanteDTO actualizarTripulanteDto)
```

Actualiza los datos de un tripulante existente. Requiere rol Administrador. Retorna 404 si el tripulante no existe.

CODIGO

```
public async Task<IActionResult> Eliminar(int id)
```

Elimina un tripulante por su identificador. Requiere rol Administrador. Retorna 404 si el tripulante no existe.

CODIGO

```
public async Task<IActionResult> SubirImagen(int id, [FromBody] SubirImagenDTO dto)
```

Sube o reemplaza la imagen de un tripulante enviada como cadena Base64. Requiere rol Administrador. Retorna 404 si el tripulante no existe.

CODIGO

```
public async Task<IActionResult> EliminarImagen(int id)
```

Elimina la imagen asociada a un tripulante. Requiere rol Administrador. Retorna 404 si el tripulante no existe.

UsuariosController

Controlador de usuarios. Expone endpoints para el registro publico de nuevos usuarios, la validacion de datos en tiempo real durante el registro, el cambio de rol por parte del administrador y la consulta del listado completo de usuarios.

CODIGO

```
public async Task<IActionResult> CrearUsuario([FromBody] CrearUsuarioDTO dto)
```

Crea un nuevo usuario en el sistema tras verificar que el correo, nombre de usuario y pasaporte no esten ya en uso. Endpoint publico, no requiere autenticacion.

CODIGO

```
public async Task<IActionResult> VerificarConstraints([FromBody] CrearUsuarioDTO dto)
```

Verifica si el correo, nombre de usuario o pasaporte del DTO ya existen en el sistema. Se usa para validacion en tiempo real mientras el usuario completa el formulario de registro. Endpoint publico, no requiere autenticacion.

CODIGO

```
public async Task<IActionResult> CambiarRol([FromBody] CambiarRolDTO dto)
```

Cambia el rol de un usuario existente. Requiere rol Administrador. Retorna el resultado de la operacion con un mensaje descriptivo del cambio realizado.

CODIGO

```
public async Task<IActionResult> ObtenerTodos()
```

Retorna el listado completo de usuarios registrados en el sistema. Requiere rol Administrador.

VotosController

Controlador de votos en comentarios. Permite a los usuarios autenticados emitir votos positivos o negativos en comentarios, quitar votos previamente emitidos y consultar su voto actual sobre un comentario especifico. Todos los endpoints requieren sesion activa.

CODIGO

```
public async Task<IActionResult> VotarComentario([FromBody] VotarComentarioDTO dto)
```

Registra un voto positivo (1) o negativo (-1) del usuario autenticado sobre un comentario. Si el usuario ya habia votado el mismo valor, el voto se anula; si era distinto, se actualiza.

CODIGO

```
public async Task<IActionResult> QuitarVoto(int comentarioId)
```

Elimina el voto del usuario autenticado sobre un comentario específico. Retorna el nuevo conteo de votos del comentario tras la eliminación.

CODIGO

```
public async Task<IActionResult> ObtenerVotoUsuario(int comentarioId)
```

Retorna el voto actual del usuario autenticado sobre un comentario específico. El valor puede ser 1 (upvote), -1 (downvote) o null si no ha votado.

VueloAgenciaController

Controlador de búsqueda de vuelos para agencias de viaje externas. Expone el endpoint de búsqueda que aplica el descuento negociado de la agencia a las tarifas retornadas. Requiere autenticación de agencia mediante AgenciaAuthMiddleware.

CODIGO

```
public async Task<IActionResult> BuscarVuelos([FromBody] BuscarVueloAgenciaDTO dto)
```

Busca vuelos disponibles según los criterios del DTO y aplica el descuento de la agencia autenticada a las tarifas retornadas. Requiere autenticación de agencia.

VuelosController

Controlador principal de vuelos del portal web. Expone endpoints públicos para buscar vuelos disponibles por criterios de origen, destino, fecha y pasajeros, y para realizar una búsqueda general por texto libre. No requiere autenticación en ningún endpoint.

CODIGO

```
public async Task<IActionResult> BusquedaGeneral([FromQuery] string query)
```

Realiza una búsqueda general de vuelos por texto libre (número de vuelo, ciudad, código de aeropuerto, etc.). Requiere al menos 2 caracteres en el término de búsqueda. Endpoint público, no requiere autenticación.

CODIGO

```
public async Task<IActionResult> BuscarVuelos([FromBody] BuscarVueloDTO dto)
```

Busca vuelos disponibles entre origen y destino para la fecha y cantidad de pasajeros indicados en el DTO. Si el usuario está autenticado, registra la búsqueda para métricas. Endpoint público, no requiere autenticación.

Services

AdminVueloService

Servicio de administracion de vuelos. Contiene la logica de negocio para crear, cancelar y consultar vuelos, asi como verificar disponibilidad de aviones y tripulantes.

CODIGO

```
public async Task<int> CrearVuelo(CrearVueloAdminDTO dto)
```

Crea un nuevo vuelo aplicando validaciones sobre numero de vuelo, aeropuertos, avion, fecha, horario, cantidad de boletos, precios y existencia de ruta entre los aeropuertos. Retorna el ID del vuelo creado.

CODIGO

```
public async Task<List<VueloHistorialDTO>> ObtenerHistorialVuelos()
```

Retorna el historial completo de vuelos registrados en el sistema, incluyendo vuelos pasados, activos y cancelados.

CODIGO

```
public async Task<bool> CancelarVuelo(int vueloId)
```

Cancela un vuelo existente dado su identificador. Valida que el ID sea mayor a cero antes de proceder con la cancelacion en el repositorio.

CODIGO

```
public async Task<HashSet<int>> ObtenerAvionesOcupados()
```

Retorna el conjunto de IDs de aviones que ya tienen un vuelo programado para la fecha, hora de salida y aeropuerto de origen indicados. Permite filtrar aviones no disponibles al momento de crear un vuelo nuevo.

CODIGO

```
public async Task<HashSet<int>> ObtenerTripulantesOcupados(DateTime fecha, TimeSpan horaSalida)
```

Retorna el conjunto de IDs de tripulantes que ya estan asignados a algun vuelo en la fecha y hora de salida indicadas. Permite evitar conflictos de asignacion.

AeropuertoService

Servicio de aeropuertos. Gestiona la logica de negocio para consultar, crear, actualizar y eliminar aeropuertos, incluyendo manejo de imagenes y fechas disponibles.

CODIGO

```
public async Task<List<AeropuertoDTO>> ObtenerAeropuertos()
```

Retorna la lista completa de aeropuertos registrados en el sistema.

CODIGO

```
public async Task<AeropuertoDTO?> ObtenerPorId(int id)
```

Busca y retorna un aeropuerto por su identificador unico. Retorna null si no existe.

CODIGO

```
public async Task<List<DateTime>> ObtenerFechasDisponibles()
```

Retorna todas las fechas para las que existe al menos un vuelo programado, sin importar la ruta.

CODIGO

```
public async Task<List<DateTime>> ObtenerFechasDisponiblesPorRuta(
```

Retorna las fechas disponibles con vuelos para una ruta especifica definida por aeropuerto de origen, destino, cantidad de personas y clase de vuelo.

CODIGO

```
public async Task<AeropuertoDTO?> Crear(CrearAeropuertoDTO crearAeropuertoDTO)
```

Crea un nuevo aeropuerto a partir del DTO recibido. Resuelve o crea el pais, ciudad y zona horaria correspondientes. Si ya existe un aeropuerto con el mismo codigo IATA, lo actualiza en lugar de crear uno nuevo para evitar duplicados.

CODIGO

```
public async Task<bool> Actualizar(int id, CrearAeropuertoDTO actualizarAeropuertoDto)
```

Actualiza los datos de un aeropuerto existente. Verifica que no existan duplicados de nombre o codigo IATA con otros aeropuertos, y resuelve la zona horaria indicada. Si se proporciona imagen nueva, la guarda junto con los demas cambios.

CODIGO

```
public async Task<bool> Eliminar(int id)
```

Elimina el aeropuerto con el identificador indicado del sistema.

CODIGO

```
public async Task GuardarImagen(int aeropuertoId, string imagenBase64)
```

Guarda o reemplaza la imagen en formato Base64 asociada al aeropuerto indicado.

CODIGO

```
public async Task EliminarImagen(int aeropuertoId)
```


Elimina la imagen asociada al aeropuerto indicado.

AgenciaService

Servicio de agencias. Gestiona la logica de negocio para crear, consultar y administrar agencias de viaje, incluyendo asignacion de usuarios webservice, descuentos, estados y URL publica de la agencia.

CODIGO

```
public async Task<AgenciaResponseDTO> CrearAgencia(CrearAgenciaDTO dto)
```

Crea una nueva agencia asignando el usuario webservice indicado en el DTO. Verifica que el usuario exista, tenga rol WebService y no tenga ya una agencia ni un hotel aliado asignados, ya que solo puede tener una entidad activa. Uso exclusivo del administrador.

CODIGO

```
public async Task<AgenciaResponseDTO> CrearAgenciaWebservice(int usuarioId, CrearAgenciaWebserviceDTO dto)
```

Permite que un usuario con rol Webservice registre su propia agencia. Verifica que no tenga ya una agencia ni un hotel aliado registrado. El ID del usuario se toma de la sesion activa.

CODIGO

```
public async Task<MiAgenciaDTO?> ObtenerMiAgencia(int usuarioId)
```

Retorna la informacion de la agencia asociada al usuario Webservice autenticado. Retorna null si el usuario aun no tiene ninguna agencia registrada.

CODIGO

```
public async Task<List<AgenciaAdminDTO>> ObtenerTodasAdmin()
```

Retorna la lista completa de agencias registradas en el sistema, incluyendo datos ampliados para uso administrativo.

CODIGO

```
public async Task<List<UsuarioWebserviceDTO>> ObtenerWebserviceSinAgencia()
```

Retorna la lista de usuarios con rol Webservice que no tienen ninguna entidad asignada (ni agencia ni hotel aliado). Util para los selectores del panel admin.

CODIGO

```
public async Task AsignarUsuario(int agenciaId, int usuarioId)
```

Asigna un usuario Webservice a una agencia existente. Verifica que el usuario exista, tenga rol Webservice y no este ya asignado a ninguna otra entidad.

CODIGO

```
public async Task ActualizarDescuento(int agenciaId, decimal descuento)
```

Actualiza el porcentaje de descuento de una agencia. El valor debe estar entre 0 y 100. Este descuento se aplica sobre el precio de los vuelos al buscar.

CODIGO

```
public async Task ActualizarEstado(int agenciaId, int estadoId)
```

Actualiza el estado de una agencia. El valor del estado debe ser un ID valido entre 1 y 3 segun el catalogo de estados de agencia.

CODIGO

```
public async Task ActualizarUrl(int agenciaId, string urlAgencia)
```

Actualiza la URL publica de una agencia desde el panel de administracion. La URL no puede estar vacia.

AsientoAgenciaService

Servicio de asientos para agencias. Gestiona la consulta y cambio de asientos en reservaciones realizadas por una agencia de viaje.

CODIGO

```
public async Task<List<AsientosVueloAgenciaDTO>> ObtenerAsientosPorReservacion(int reservacionId, int agenciaId)
```

Retorna la lista de asientos asignados en una reservacion especifica perteneciente a la agencia indicada.

CODIGO

```
public async Task CambiarAsiento(int boletoId, string nuevoAsiento, int agenciaId)
```

Cambia el asiento de un boleto perteneciente a una reservacion de la agencia. Valida que el nuevo numero de asiento no sea vacio y lo convierte a mayusculas.

AsientoService

Servicio de asientos para usuarios registrados. Gestiona la consulta del mapa de asientos de un vuelo y el cambio de asiento en boletos pertenecientes al usuario autenticado.

CODIGO

```
public async Task<AsientosVueloDTO> ObtenerAsientosVuelo(int vueloId, int usuarioId)
```

Retorna el mapa de asientos de un vuelo especifico, indicando cuales estan disponibles y cuales ya han sido reservados por el usuario autenticado.

CODIGO

```
public async Task CambiarAsiento(int boletoId, string nuevoAsiento, int usuarioId)
```

Cambia el asiento asignado a un boleto del usuario. Valida que el nuevo asiento no sea vacio y lo normaliza a mayusculas antes de persistirlo.

AuthService

Servicio de autenticacion. Verifica las credenciales del usuario contra la base de datos y retorna la informacion de sesion si la autenticacion es exitosa.

CODIGO

```
public async Task<LoginResponseDto?> Login(LoginRequestDto request)
```

Autentica al usuario usando correo o nombre de usuario y contrasena. Verifica el hash de contrasena con BCrypt. Retorna los datos de sesion incluyendo rol del usuario, o null si las credenciales son incorrectas.

AvionService

Servicio de aviones. Gestiona la logica de negocio para registrar, consultar, actualizar y eliminar aviones de la flota, incluyendo el manejo de imagenes.

CODIGO

```
public async Task<List<AvionDTO>> ObtenerTodos()
```

Retorna la lista completa de aviones registrados en el sistema, incluyendo marca, modelo, capacidad e imagen en Base64.

CODIGO

```
public async Task<AvionDTO?> ObtenerPorId(int id)
```

Busca y retorna un avion por su identificador unico. Retorna null si el avion no existe en el sistema.

CODIGO

```
public async Task<AvionDTO> Crear(CrearAvionDTO crearAvionDto)
```

Crea un nuevo avion en el sistema a partir del DTO recibido. Si se incluye imagen en Base64, la guarda de manera independiente en el repositorio. Retorna el DTO del avion recién creado con su ID asignado.

CODIGO

```
public async Task<bool> Actualizar(int id, CrearAvionDTO actualizarAvionDto)
```

Actualiza los datos de un avion existente. Si se proporciona una nueva imagen en Base64, tambien la actualiza en el repositorio.

CODIGO

```
public async Task<bool> Eliminar(int id)
```

Elimina el avion con el identificador indicado del sistema.

CODIGO

```
public async Task GuardarImagen(int avionId, string imagenBase64)
```

Guarda o reemplaza la imagen en formato Base64 asociada al avion indicado.

CODIGO

```
public async Task EliminarImagen(int avionId)
```

Elimina la imagen asociada al avion indicado.

Busquedatemporalservice

Servicio de búsquedas temporales en memoria. Almacena el contexto de búsqueda de vuelos de forma temporal usando un diccionario concurrente para soportar multiples solicitudes simultaneas. Las entradas expiradas (mayores a 1 hora) se eliminan automaticamente en cada escritura.

CODIGO

```
public string GuardarBusqueda(BusquedaVuelo busqueda)
```

Guarda el contexto de una búsqueda de vuelo en memoria y retorna su identificador unico. Antes de guardar, elimina todas las búsquedas con mas de una hora de antigüedad.

ComentarioService

Servicio de comentarios. Gestiona la logica de negocio para crear comentarios con calificacion sobre rutas, publicar respuestas a comentarios y consultar comentarios con informacion de votos por usuario.

CODIGO

```
public async Task<ComentarioDTO> CrearComentarioRuta(int usuarioId, CrearComentarioRutaDTO dto)
```

Crea un comentario con calificacion de estrellas sobre una ruta especifica. Valida que la cantidad de estrellas este entre 1 y 5 y que el contenido no este vacio ni supere los 500 caracteres.

CODIGO

```
public async Task<ComentarioDTO> CrearRespuesta(int usuarioId, CrearRespuestaDTO dto)
```

Crea una respuesta a un comentario existente. Valida que el contenido no este vacio y no supere los 500 caracteres.

CODIGO

```
public async Task<List<ComentarioConVotoDTO>> ObtenerTodosConVoto(int usuarioId)
```

Retorna todos los comentarios del sistema incluyendo el voto que el usuario autenticado ha emitido sobre cada uno, si lo hay.

CODIGO

```
public async Task<List<ComentarioDTO>> ObtenerComentariosPorUsuario(int usuarioId)
```

Retorna todos los comentarios publicados por un usuario específico.

CODIGO

```
public async Task<List<ComentarioConVotoDTO>> ObtenerComentariosRutaConVoto(int rutaId, int usuarioId)
```

Retorna los comentarios de una ruta específica incluyendo el estado del voto del usuario autenticado en cada comentario.

CODIGO

```
public async Task<List<ComentarioDTO>> ObtenerComentariosPorRuta(int rutaId)
```

Retorna todos los comentarios asociados a una ruta específica sin información de votos.

ConfirmarReservacionAgenciaService

Servicio de confirmación de reservaciones para agencias. Gestiona la lógica de negocio para confirmar una reservación pendiente de una agencia, validando los datos fiscales requeridos antes de proceder con el pago y emisión de boletos.

CODIGO

```
public async Task<ConfirmacionAgenciaDTO> ConfirmarReservacion(
```

Confirma una reservación pendiente de una agencia aplicando los datos fiscales del DTO. Valida que el NIT y el código postal no estén vacíos antes de delegar al repositorio. Retorna los datos de confirmación con la información de la factura generada.

DownService

Servicio de votos en comentarios. Gestiona la lógica de negocio para emitir, quitar y consultar votos (upvote/downvote) de los usuarios sobre comentarios de rutas.

CODIGO

```
public async Task<ResultadoVotoDTO> VotarComentario(int usuarioId, VotarComentarioDTO dto)
```

Registra o actualiza el voto de un usuario sobre un comentario. El valor del voto debe ser 1 para upvote o -1 para downvote. Retorna el nuevo conteo de votos positivos y negativos del comentario.

CODIGO

```
public async Task<ResultadoVotoDTO> QuitarVoto(int usuarioId, int comentarioId)
```

Elimina el voto que el usuario habia emitido sobre un comentario especifico. Retorna el nuevo conteo de votos del comentario tras la eliminacion.

CODIGO

```
public async Task<int?> ObtenerVotoUsuario(int usuarioId, int comentarioId)
```

Retorna el valor del voto que el usuario ha emitido sobre un comentario (1, -1 o null). Retorna null si el usuario no ha votado en ese comentario.

FacturaService

Servicio de facturacion. Gestiona la logica de negocio para procesar la compra de una reservacion, validando datos fiscales y el formato de tarjeta de pago antes de generar la factura y emitir los boletos.

CODIGO

```
public async Task<CompraRealizadaDTO> ComprarReservacion(
```

Procesa la compra de una reservacion existente. Valida que el NIT y codigo postal esten presentes, verifica el formato de la tarjeta de pago mediante TarjetaHelper (sin almacenar datos de la tarjeta) y delega la creacion de factura y boletos al repositorio.

GestionReservacionService

Servicio de gestion de reservaciones. Permite a los usuarios consultar, cancelar y obtener resúmenes de sus reservaciones, además de enviar comprobantes por correo electrónico. También expone métodos de apoyo para el flujo de agencias.

CODIGO

```
public async Task<List<ReservacionDetalleDTO>> ObtenerMisReservaciones(int usuarioId)
```

Retorna la lista de todas las reservaciones del usuario autenticado, incluyendo detalle de vuelos, pasajeros y estado de cada reservacion.

CODIGO

```
public async Task<ReservacionDetalleDTO> ObtenerDetalleReservacion(int reservacionId, int usuarioId)
```

Retorna el detalle completo de una reservacion especifica perteneciente al usuario. Lanza excepcion si la reservacion no existe o no pertenece al usuario indicado.

CODIGO

```
public async Task<ResumenReservacionesDTO> ObtenerResumen(int usuarioId)
```

Retorna un resumen estadístico de las reservaciones del usuario, incluyendo totales por estado y monto acumulado de compras.

CODIGO

```
public async Task CancelarReservacion(int reservacionId, int usuarioId, string motivo)
```

Cancela una reservacion activa del usuario. Registra el motivo de cancelacion y libera los asientos y boletos asociados segun la logica del repositorio.

CODIGO

```
public async Task EnviarComprobanteEmail(int reservacionId, int usuarioId)
```

Envia al correo del usuario un comprobante en formato HTML con el detalle completo de su reservacion. El envio se realiza de forma asincrona mediante EmailHelper.

CODIGO

```
public async Task<int> ObtenerUsuarioWebIdDeAgencia(int agenciaId)
```

Retorna el ID del usuario Webservice asociado a la agencia indicada. Utilizado para determinar el responsable de reservaciones hechas por agencias.

CODIGO

```
public async Task<PuedeCancelarDTO> PuedeCancelar(int reservacionId, int usuarioId)
```

Verifica si una reservacion puede ser cancelada por el usuario en el momento actual, considerando politicas de tiempo minimo antes del vuelo y el estado actual de la reservacion.

HandshakeHotelService

Servicio encargado de iniciar el proceso de handshake de autenticacion con un hotel aliado externo. Genera un token de entrada que identifica a la aerolinea, lo envia al hotel junto con la URL publica de la aerolinea y almacena el token de sesion resultante en la columna TokenHASH del registro HotelAliado. Cuando el backend corre dentro de Docker y la URL del hotel contiene 'localhost', se sustituye ese host por el valor de HANDSHAKEHOSTOVERRIDE (host.docker.internal) para poder alcanzar el servidor Java corriendo en la maquina anfitriona.

CODIGO

```
public async Task<string> IniciarHandshake(int hotelId)
```

Ejecuta el flujo completo de handshake con un hotel aliado. Obtiene la URL del hotel desde la base de datos sin filtrar por estado, aplica la sustitucion de host si es necesaria para entornos Docker, genera un token de entrada, lo envia al hotel, recibe el token de sesion y lo guarda en HotelAliado.TokenHASH. Si el hotel no existe, no tiene URL configurada, falla la llamada HTTP o falla el guardado en base de datos.

- Param hotelId - ID del registro HotelAliado con quien iniciar el handshake.
- Returns - El token de sesion recibido del hotel aliado.

- @throws Exception

HandshakeService

Servicio de handshake entre la aerolinea y agencias externas. Gestiona el intercambio de tokens de autenticacion para establecer una sesion segura con una agencia registrada identificada por su URL.

CODIGO

```
public async Task<HandshakeResponseDTO> ProcesarHandshake(HandshakeRequestDTO dto)
```

Procesa la solicitud de handshake de una agencia externa. Busca la agencia por su URL, genera un token de salida y guarda ambos tokens (entrada y salida) en la base de datos. Retorna el token de salida que la agencia debe usar en solicitudes posteriores.

HotelAliadoService

Servicio de hoteles aliados. Gestiona la busqueda dinamica de hoteles consultando cada aliado activo con su propia URL y TokenHASH, el registro y consulta de hoteles por usuarios Webservice, y la administracion completa desde el panel de administracion.

CODIGO

```
public async Task<List<HotelAliadoDTO>> BuscarHoteles(BusquedaHotelesDTO dto)
```

Obtiene todos los hoteles aliados activos de la base de datos y consulta cada uno con su propia URL y token. Agrega todos los resultados en una sola lista. Si un aliado falla, se omite sin interrumpir la busqueda en los demas.

- Param dto - Criterios de busqueda: ciudad, pais, fechas y personas.
- Returns - Lista combinada de hoteles de todos los aliados activos.

CODIGO

```
public async Task<MiHotelDTO> CrearHotelWebservice(int usuarioId, CrearHotelWebserviceDTO dto)
```

Permite que un usuario con rol Webservice registre su propio hotel aliado. Verifica que no tenga ya un hotel ni una agencia registrada. El ID del usuario se toma de la sesion activa.

CODIGO

```
public async Task<MiHotelDTO?> ObtenerMiHotel(int usuarioId)
```

Retorna la informacion del hotel aliado asociado al usuario Webservice autenticado. Retorna null si el usuario aun no tiene ningun hotel registrado.

CODIGO

```
public async Task<List<HotelAdminDTO>> ObtenerTodosAdmin()
```

Retorna la lista completa de hoteles aliados con datos del usuario asignado. Destinado al uso exclusivo del panel de administracion.

CODIGO

```
public async Task<HotelAdminDTO> CrearHotelAdmin(CrearHotelAdminDTO dto)
```

Crea un nuevo hotel aliado desde el panel de administracion, asignandolo al usuario Webservice indicado. Verifica que el usuario exista, tenga rol Webservice y no tenga ya ninguna otra entidad asignada (agencia o hotel).

CODIGO

```
public async Task ActualizarEstado(int hotelId, int estadoId)
```

Actualiza el estado de un hotel aliado. El EstadoId debe ser un valor valido del catalogo EstadoAliado (1=Activo, 2=Inactivo, 3=Suspendido).

CODIGO

```
public async Task AsignarUsuario(int hotelId, int usuarioId)
```

Asigna un usuario Webservice a un hotel aliado existente. Verifica que el usuario exista, tenga rol Webservice y no este ya vinculado a ninguna entidad.

CODIGO

```
public async Task ActualizarUrls(int hotelId, string url, string urlParaUsuario)
```

Actualiza la URL de la API y la URL publica de un hotel aliado. Ninguna de las dos URLs puede estar vacia.

MetricasService

Servicio de metricas del sistema. Provee datos estadisticos sobre busquedas de vuelos, rutas mas solicitadas y distribucion por tipo, con soporte para filtros de fecha. Tambien permite exportar el listado completo sin paginacion.

CODIGO

```
public async Task<MetricasResumenDTO> ObtenerResumen(string? fechaDesde, string? fechaHasta)
```

Retorna un resumen de las metricas del sistema en el rango de fechas indicado. Incluye totales de busquedas, usuarios activos y reservaciones generadas.

CODIGO

```
public async Task<List<BusquedasPorDiaDTO>> ObtenerBusquedasPorDia(
```

Retorna la cantidad de busquedas realizadas por dia en el rango de fechas indicado. Util para graficar la evolucion de la demanda a lo largo del tiempo.

CODIGO

```
public async Task<List<RutaMasBuscadaDTO>> ObtenerRutasMasBuscadas(
```

Retorna el listado de rutas con mayor cantidad de búsquedas en el rango de fechas, filtrado opcionalmente por tipo de búsqueda (directo, con escala, etc.).

CODIGO

```
public async Task<List<BusquedasPorTipoDTO>> ObtenerBusquedasPorTipo()
```

Retorna la distribución de búsquedas agrupadas por tipo en el rango de fechas indicado. Permite identificar que tipo de viaje es más demandado por los usuarios.

CODIGO

```
public async Task<ListadoBusquedasDTO> ObtenerListado(MetricasFiltroDTO filtro)
```

Retorna un listado paginado de búsquedas aplicando los filtros del objeto de filtro recibido. Incluye información de usuario, fechas y parámetros de cada búsqueda.

CODIGO

```
public async Task<ListadoBusquedasDTO> ObtenerListadoCompleto(MetricasFiltroDTO filtro)
```

Retorna el listado completo de búsquedas sin paginación, aplicando solo los filtros de fecha, tipo y usuario. Pensado para exportación de datos en reportes.

NacionalidadService

Servicio de nacionalidades. Expone la lógica de negocio para consultar el catálogo de nacionalidades disponibles en el sistema.

CODIGO

```
public async Task<List<NacionalidadDto>> ObtenerTodas()
```

Retorna la lista completa de nacionalidades disponibles mapeadas a su DTO, incluyendo ID y nombre de cada una.

PdfService

Servicio de generación de documentos PDF. Convierte contenido HTML a bytes de PDF. Temporalmente retorna un array vacío mientras la librería nativa wkhtmltopdf no esté disponible en el entorno de desarrollo.

CODIGO

```
public byte[] GenerarPdf(string html)
```

Genera un archivo PDF a partir del contenido HTML recibido. Retorna un array vacío si la librería nativa no está disponible.

PerfilService

Servicio de perfil de usuario. Gestiona la consulta de datos personales, la actualizacion del numero de telefono y el cambio de contraseña con verificacion del hash actual almacenado.

CODIGO

```
public async Task<PerfilDTO?> ObtenerPerfil(int usuarioId)
```

Retorna los datos del perfil del usuario autenticado. Retorna null si el usuario no existe en el sistema.

CODIGO

```
public async Task<(bool exito, string mensaje)> ActualizarTelefono(int usuarioId, string telefono)
```

Actualiza el numero de telefono del usuario. Valida que el campo no este vacio. Retorna una tupla con un indicador de exito y un mensaje descriptivo del resultado.

CODIGO

```
public async Task<(bool exito, string mensaje)> CambiarContraseña(int usuarioId, CambiarContraseñaDTO dto)
```

Cambia la contraseña del usuario tras verificar que la contraseña actual sea correcta y que la nueva tenga al menos 8 caracteres. Hashea la nueva contraseña antes de guardarla. Retorna una tupla con un indicador de exito y un mensaje descriptivo del resultado.

ReservacionAgenciaService

Servicio de reservaciones para agencias. Gestiona la logica de negocio para que una agencia pueda crear reservaciones con descuento, expirarlas manualmente y agregar pasajeros con sus datos de pasaporte.

CODIGO

```
public async Task<ReservacionCreadaDTO> CrearReservacion(CrearReservacionDTO dto, int agenciaId)
```

Crea una nueva reservacion para la agencia indicada. Obtiene el porcentaje de descuento configurado para la agencia y lo aplica al momento de crear la reservacion en el repositorio.

CODIGO

```
public async Task ExpirarReservacion(int reservacionId, int agenciaId)
```

Expira manualmente una reservacion pendiente de la agencia. Verifica que la reservacion pertenezca a la agencia y que este en estado pendiente antes de proceder.

CODIGO

```
public async Task AgregarPasajeros(AgregarPasajerosDTO dto, int agenciaId)
```

Agrega la lista de pasajeros a una reservacion existente de la agencia. Valida que cada pasajero tenga numero de pasaporte y que este contenga solo digitos.

ReservacionService

Servicio de reservaciones para usuarios registrados. Gestiona la creacion de reservaciones y la asignacion de pasajeros, validando que el usuario este autenticado y que los datos de pasaporte sean correctos antes de persistirlos.

CODIGO

```
public async Task<ReservacionCreadaDTO> CrearReservacion(CrearReservacionDTO dto, int? usuarioId)
```

Crea una nueva reservacion para el usuario autenticado. Requiere que el usuarioId no sea nulo; de lo contrario lanza una excepcion indicando que debe iniciar sesion.

CODIGO

```
public async Task AgregarPasajeros(AgregarPasajerosDTO dto)
```

Agrega la lista de pasajeros a una reservacion existente. Valida que cada pasajero tenga numero de pasaporte y que este contenga unicamente digitos numericos.

ReservasCleanupService

Servicio en segundo plano para el mantenimiento automatico de reservaciones y vuelos. Se ejecuta como BackgroundService y realiza tres tareas periodicas: liberar reservas expiradas, completar reservaciones cuyo vuelo ya paso, y actualizar el estado de los vuelos (en transcurso y finalizados) cada 30 ciclos de ejecucion (30 minutos).

CODIGO

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
```

Ejecuta el servicio en segundo plano de forma continua. Libera reservas expiradas, completa reservaciones cuyo vuelo ya paso y actualiza el estado de los vuelos cada 30 ciclos (30 minutos).

RutaAgenciaService

Servicio de rutas para agencias. Provee acceso al catalogo completo de rutas disponibles que las agencias pueden usar al buscar y reservar vuelos.

CODIGO

```
public async Task<List<RutaAgenciaDTO>> ObtenerTodasLasRutas()
```

Retorna la lista completa de rutas disponibles en el sistema con los datos necesarios para que una agencia realice busquedas de vuelos.

RutaService

Servicio de rutas aereas. Gestiona la logica de negocio para consultar, crear y actualizar rutas entre aeropuertos, incluyendo el calculo de hora de llegada con conversion de zonas horarias IANA y Windows.

CODIGO

```
public async Task<List<RutaDTO>> ObtenerTodas()
```

Retorna la lista completa de rutas registradas en el sistema.

CODIGO

```
public async Task<bool> ActualizarDuracion(int id, int duracionMinutos)
```

Actualiza la duracion estimada en minutos de una ruta existente. Valida que el valor sea mayor a 0 y no supere los 10,000 minutos.

CODIGO

```
public async Task<CalculoLlegadaResponseDTO> CalcularLlegada(CalculoLlegadaRequestDTO request)
```

Calcula la hora y fecha de llegada estimadas dado el aeropuerto de origen, destino, fecha y hora de salida. Aplica conversion de zonas horarias si ambos aeropuertos tienen zona configurada; de lo contrario usa calculo directo sin conversion.

CODIGO

```
public async Task<bool> ExisteRuta(int origenId, int destinoId)
```

Verifica si ya existe una ruta registrada entre los aeropuertos de origen y destino indicados.

CODIGO

```
public async Task<(bool creada, int rutaId, string mensaje)> CrearRuta(
```

Crea una nueva ruta entre dos aeropuertos con la duracion estimada en minutos. Valida que origen y destino sean distintos, que la duracion sea valida y que la ruta no exista previamente. Retorna una tupla con el resultado, el ID y un mensaje.

TokenHotelService

Servicio que solicita un token de alianza a un hotel aliado especifico. Busca el hotel en la BD de aerolineas por su ID, llama a su endpoint de generacion de token y retorna el resultado al frontend.

CODIGO

```
public async Task<TokenHotelResponseDTO> SolicitarToken(int aliadoId, TokenHotelRequestDTO dto)
```

Busca el hotel aliado por su ID en la BD, llama a su endpoint POST /aerolinea/token con el TokenHASH y retorna el token generado junto con la URL de redireccion para el usuario.

- Param aliadold - ID del registro HotelAliado en la BD de aerolineas.
- Param dto - Ciudad y pais destino del pasajero.
- Returns - TokenHotelResponseDTO con token, URL y fecha de expiracion.
- Throws Exception - Si el aliado no existe, no esta activo o el hotel responde con error.

TripulacionService

Servicio de tripulacion. Gestiona la logica de negocio para registrar, consultar, actualizar y eliminar tripulantes de vuelo, incluyendo la asignacion de roles, manejo de imagenes y consulta del catalogo de roles disponibles.

CODIGO

```
public async Task<List<TripulanteDTO>> ObtenerTodos()
```

Retorna la lista completa de tripulantes registrados en el sistema. Por cada tripulante resuelve el nombre del rol mediante una consulta adicional al repositorio y construye el DTO con el nombre completo concatenado.

CODIGO

```
public async Task<TripulanteDTO?> ObtenerPorId(int id)
```

Busca y retorna los datos de un tripulante especifico por su ID, incluyendo el nombre del rol resuelto desde el repositorio. Retorna null si el tripulante no existe.

CODIGO

```
public async Task<TripulanteDTO> Crear(CrearTripulanteDTO crearTripulanteDTO)
```

Crea un nuevo tripulante a partir del DTO recibido y retorna su DTO con el ID asignado.

CODIGO

```
public async Task<bool> Actualizar(int id, CrearTripulanteDTO actualizarTripulanteDto)
```

Actualiza los datos de un tripulante existente usando el ID y el DTO proporcionados.

CODIGO

```
public async Task<bool> Eliminar(int id)
```

Elimina el tripulante con el identificador indicado del sistema.

CODIGO

```
public async Task GuardarImagen(int tripulanteId, string imagenBase64)
```

Guarda o reemplaza la imagen en formato Base64 del tripulante indicado.

CODIGO

```
public async Task EliminarImagen(int tripulanteId)
```

Elimina la imagen asociada al tripulante indicado.

CODIGO

```
public async Task<List<RolTripulacion>> ObtenerRoles()
```

Retorna el catalogo completo de roles de tripulacion disponibles en el sistema, como piloto, copiloto, auxiliar de vuelo, etc.

UsuarioService

Servicio de usuarios. Gestiona el registro de nuevos usuarios con validaciones de formato, la verificacion de campos unicos, el cambio de rol por parte del administrador y la consulta del listado completo de usuarios del sistema.

CODIGO

```
public async Task CrearUsuario(CrearUsuarioDTO dto)
```

Registra un nuevo usuario en el sistema. Valida formato de pasaporte y telefono, resuelve o crea el pais y ciudad correspondientes, hashea la contrasena y asigna el rol de cliente por defecto. Tambien guarda las nacionalidades si se proveen y envia un correo de bienvenida de forma no bloqueante.

CODIGO

```
public async Task<RegisterConstraint> VerificarConstraints(CrearUsuarioDTO dto)
```

Verifica si ya existe algun usuario con el mismo correo, nombre de usuario o pasaporte. Retorna un objeto con los campos que generarian conflicto de unicidad.

CODIGO

```
public async Task<(bool exito, string mensaje)> CambiarRol(CambiarRolDTO dto)
```

Cambia el rol de un usuario existente. Verifica que tanto el usuario como el rol indicados existan antes de aplicar el cambio. Retorna una tupla con resultado y mensaje.

CODIGO

```
public async Task<List<object>> ObtenerTodos()
```

Retorna la lista de todos los usuarios registrados en el sistema como objetos anonimos, incluyendo ID, nombre, apellido, correo, username, ID de rol y nombre del rol.

VueloAgenciaService

Servicio de búsqueda de vuelos para agencias. Resuelve la ciudad y aeropuerto a partir del nombre de país y ciudad provistos por la agencia, aplica el descuento configurado para la agencia y retorna vuelos directos y con escala con precios ajustados.

CODIGO

```
public async Task<ResultadoBusquedaDTO> BuscarVuelos(BuscarVueloAgenciaDTO dto, int agenciaId)
```

Busca vuelos disponibles para una agencia resolviendo primero los aeropuertos de origen y destino a partir de los nombres de país y ciudad. Obtiene el descuento de la agencia, busca vuelos directos y con escala, y aplica el factor de descuento a todos los precios antes de retornar los resultados.

VueloService

Servicio de vuelos para usuarios. Gestiona la búsqueda de vuelos directos y con escala, el registro de búsquedas para métricas y la aplicación de filtros de precio en memoria.

CODIGO

```
public async Task<List<VueloDetalleDTO>> BusquedaGeneral(string query)
```

Realiza una búsqueda general de vuelos por texto libre. Retorna una lista de vuelos cuyo número, origen o destino coincidan con la consulta.

CODIGO

```
public async Task<ResultadoBusquedaDTO> BuscarVuelos(BuscarVueloDTO dto, int? usuarioId)
```

Busca vuelos disponibles entre dos aeropuertos en una fecha y con los filtros indicados. Registra la búsqueda en la base de datos para métricas. Retorna vuelos directos y con escala, ambos filtrados por rango de precio si se especifica.

Repositories

AdminVueloRepository

Repositorio de administracion de vuelos. Gestiona la creacion, cancelacion, historial y disponibilidad de aviones y tripulacion en la base de datos.

CODIGO

```
public async Task<int> CrearVuelo(CrearVueloAdminDTO dto)
```

Crea un nuevo vuelo en la base de datos dentro de una transaccion. Verifica o crea la ruta, calcula la hora de llegada segun zonas horarias, valida la capacidad del avion y asigna la tripulacion indicada. Retorna el ID del vuelo creado.

CODIGO

```
public async Task<List<VueloHistorialDTO>> ObtenerHistorialVuelos()
```

Obtiene el historial completo de vuelos con informacion de ruta, avion, estado, boletos vendidos y zonas horarias. Los resultados se ordenan por fecha descendente.

CODIGO

```
public async Task<bool> CancelarVuelo(int vueloId)
```

Cancela un vuelo activo o en curso. Marca el vuelo con estado Cancelado, cancela todos los boletos activos y actualiza las reservaciones relacionadas al estado Cancelado dentro de una transaccion atomica.

CODIGO

```
public async Task<HashSet<int>> ObtenerAvionesOcupados()
```

Devuelve los IDs de aviones que no estan disponibles para un vuelo nuevo. Un avion no esta disponible si: a) Todavia esta en vuelo al momento de la nueva salida. b) Aterrizo en el mismo aeropuerto de origen hace menos de 24 horas. c) Aterrizo en un aeropuerto diferente hace menos de 48 horas. FechaLlegada + HoraLlegada define el momento real de aterrizaje.

CODIGO

```
public async Task<HashSet<int>> ObtenerTripulantesOcupados(DateTime fecha, TimeSpan horaSalida)
```

Devuelve los IDs de tripulantes que no estan disponibles para la fecha y hora indicada. Un tripulante no esta disponible si: a) Tiene un vuelo asignado el mismo dia con estado activo. b) Su vuelo mas reciente finaliza menos de 24 horas antes de la salida solicitada. FechaLlegada + HoraLlegada define el momento real de fin del vuelo.

AeropuertoRepository

Repositorio de aeropuertos. Gestiona consultas, creacion, actualizacion, eliminacion e imagenes de aeropuertos, así como la resolucio

```
CODIGO
public async Task<List<AeropuertoDT0>> ObtenerTodos()
```

Retorna la lista completa de aeropuertos con ciudad, pais, imagen y zona horaria IANA. Ordenados alfabeticamente por nombre.

```
CODIGO
public async Task<AeropuertoDT0> ObtenerPorId(int id)
```

Retorna un aeropuerto especifico con todos sus datos de ciudad, pais, imagen y zona horaria. Retorna null si no existe el ID dado.

```
CODIGO
public async Task<string?> VerificarDuplicado(string nombre, string codigo, int? excludeId = null)
```

Verifica si ya existe un aeropuerto con el mismo nombre o codigo IATA. Retorna el nombre del campo duplicado ('nombre' o 'codigo') o null si no hay conflicto. El parametro excludeId permite ignorar el propio registro al editar.

```
CODIGO
public async Task<int?> ObtenerIdPorCodigo(string codigo)
```

Busca el ID de un aeropuerto por su codigo IATA. Retorna null si no existe.

```
CODIGO
public async Task<int> Crear(Aeropuerto aeropuerto)
```

Inserta un nuevo aeropuerto en la base de datos. Si la columna ZonaHorariaID ya existe en el esquema, la incluye en la insercion. Retorna el ID generado.

```
CODIGO
public async Task<bool> Actualizar(Aeropuerto aeropuerto)
```

Actualiza los datos de un aeropuerto existente. Incluye ZonaHorariaID si la columna existe en el esquema actual. Retorna true si se afecto al menos una fila.

```
CODIGO
public async Task<bool> Eliminar(int id)
```

Elimina un aeropuerto y su imagen asociada de la base de datos. Retorna true si se elimino correctamente.

```
CODIGO
public async Task<int?> ObtenerOCrearZonaHoraria(string? nombreIana)
```

Busca una zona horaria IANA en el catalogo por nombre. Si no existe la crea. Retorna null si el nombre proporcionado es nulo o vacio.

CODIGO

```
public async Task GuardarImagen(int aeropuertoId, string imagenBase64)
```

Guarda o actualiza la imagen en Base64 de un aeropuerto. Usa un patron UPSERT: actualiza si ya existe registro, inserta si no.

CODIGO

```
public async Task EliminarImagen(int aeropuertoId)
```

Elimina la imagen asociada a un aeropuerto segun su ID.

CODIGO

```
public async Task<List<DateTime>> ObtenerFechasConVuelos()
```

Retorna las fechas futuras que tienen al menos un vuelo programado, ordenadas de forma ascendente. Se usa para resaltar días disponibles en el calendario.

CODIGO

```
public async Task<List<DateTime>> ObtenerFechasConVuelosPorRuta(
```

Retorna las fechas con vuelos disponibles para una ruta especifica, considerando cantidad de pasajeros, clase y numero maximo de escalas. Utiliza BFS por capas para incluir fechas de itinerarios con conexiones, igual que VueloRepository.

CODIGO

```
public async Task<List<CiudadDTO>> ObtenerCiudades()
```

Retorna la lista de ciudades con su pais, ordenadas por pais y ciudad. Incluye un campo NombreCompleto con el formato "Ciudad, Pais".

CODIGO

```
public async Task<int> ObtenerOCrearPais(string nombrePais)
```

Busca un pais por nombre. Si no existe lo crea y retorna su ID. Se usa al registrar aeropuertos con paises nuevos.

CODIGO

```
public async Task<int> ObtenerOCrearCiudad(string nombreCiudad, int paisId)
```

Busca una ciudad por nombre y pais. Si no existe la crea y retorna su ID. Se usa al registrar aeropuertos con ciudades nuevas.

CODIGO

```
public async Task<int> ObtenerIdPorCiudad(int ciudadId, SqlConnection connection)
```

Obtiene el ID del primer aeropuerto registrado para una ciudad dada. Lanza una excepcion si la ciudad no tiene aeropuertos configurados.

AgenciaRepository

Repositorio de agencias. Gestiona la creacion, consulta, actualizacion y autentificacion de agencias de viaje, incluyendo la administracion de tokens, descuentos, estados, URL y usuarios webservice asociados.

CODIGO

```
public async Task<int> ObtenerRolUsuario(int usuarioId)
```

Consulta el RolID del usuario especificado. Retorna 0 si el usuario no existe.

CODIGO

```
public async Task<bool> UsuarioYaTieneAgencia(int usuarioId)
```

Verifica si el usuario webservice dado ya tiene una agencia asignada. Retorna true si existe al menos una agencia para ese usuario.

CODIGO

```
public async Task<bool> UsuarioYaTieneHotelAliado(int usuarioId)
```

Verifica si el usuario webservice dado ya tiene un hotel aliado registrado. Se usa para impedir que un usuario tenga tanto una agencia como un hotel. Retorna true si existe al menos un hotel para ese usuario.

CODIGO

```
public async Task<AgenciaResponseDTO> CrearAgencia(CrearAgenciaDTO dto)
```

Crea una nueva agencia en la base de datos con estado Activo e incluye la URL publica. Los tokens se dejan vacios ya que se generan automaticamente al establecer la conexion. Retorna el DTO con los datos de la agencia creada incluyendo su ID generado.

CODIGO

```
public async Task<bool> GuardarTokens(int agenciaId, string tokenEntrada, string tokenSalida)
```

Actualiza los tokens de autentificacion (entrada y salida) de una agencia. Retorna true si se actualizo al menos una fila.

CODIGO

```
public async Task<int?> ObtenerAgenciaIdPorTokenEntrada(string tokenEntrada)
```

Busca el ID de la agencia a partir de su token de entrada. Retorna null si no se encuentra ninguna agencia con ese token.

```
CODIGO
public async Task<int?> ObtenerAgenciaIdPorURL(string urlAgencia)
```

Busca el ID de la agencia a partir de su URL. Retorna null si no existe.

```
CODIGO
public async Task<AgenciaIdentidad?> ObtenerAgenciaPorToken(string token)
```

Obtiene la identidad basica de una agencia (ID, nombre y URL) a partir de su token de autenticacion de entrada. Retorna null si no existe.

```
CODIGO
public async Task<decimal> ObtenerDescuento(int agenciaId)
```

Retorna el porcentaje de descuento configurado para la agencia indicada. Retorna 0 si no se encuentra la agencia.

```
CODIGO
public async Task<MiAgenciaDT0?> ObtenerAgenciaPorUsuarioId(int usuarioId)
```

Retorna los datos de la agencia del usuario webservice indicado, incluyendo la URL. Retorna null si el usuario no tiene ninguna agencia asignada.

```
CODIGO
public async Task<List<AgenciaAdminDT0>> ObtenerTodasAdmin()
```

Retorna el listado completo de agencias con los datos del usuario webservice asignado, incluyendo la URL publica. Destinado al uso exclusivo del panel de administracion.

```
CODIGO
public async Task<List<UsuarioWebserviceDT0>> ObtenerWebserviceSinAgencia()
```

Retorna la lista de usuarios con rol Webservice que no tienen ninguna entidad asignada (ni agencia ni hotel aliado). Se usa en el panel de administracion para los selectores de asignacion de ambos tipos de entidad.

```
CODIGO
public async Task<bool> AsignarUsuarioAAgencia(int agenciaId, int usuarioId)
```

Asigna o reasigna un usuario webservice a una agencia existente. Retorna true si la actualizacion fue exitosa.

```
CODIGO
public async Task<bool> ActualizarDescuento(int agenciaId, decimal descuento)
```

Actualiza el porcentaje de descuento de una agencia. Retorna true si la actualizacion fue exitosa.

CODIGO

```
public async Task<bool> ActualizarEstado(int agenciaId, int estadoId)
```

Actualiza el estado de una agencia (activa, suspendida, etc.). Retorna true si la actualizacion fue exitosa.

CODIGO

```
public async Task<bool> ActualizarUrl(int agenciaId, string urlAgencia)
```

Actualiza la URL publica de una agencia desde el panel de administracion. Retorna true si la actualizacion fue exitosa.

AsientoAgenciaRepository

Repositorio de asientos para agencias de viaje. Permite consultar el mapa de asientos de una reservacion y cambiar asientos de boletos dentro del contexto de una agencia autenticada.

CODIGO

```
public async Task<List<AsientosVueloAgenciaDTO>> ObtenerAsientosPorReservacion(int reservacionId, int agenciaId)
```

Retorna el mapa de asientos de todos los vuelos de una reservacion de agencia. Incluye asientos ocupados, boletos propios de la agencia y el layout dinamico calculado segun la cantidad de boletos por clase en cada vuelo. Lanza una excepcion si la reservacion no pertenece a la agencia.

CODIGO

```
public async Task CambiarAsiento(int boletoId, string nuevoAsiento, int agenciaId)
```

Cambia el asiento de un boleto perteneciente a una reservacion de agencia. Verifica propiedad, validez del asiento en el layout del vuelo, disponibilidad con bloqueo pesimista y regla de clase antes de aplicar el cambio.

AsientoRepository

Repositorio de asientos para usuarios. Gestiona la consulta del mapa de asientos de un vuelo y el cambio de asiento de un boleto pendiente, verificando propiedad, disponibilidad con bloqueo pesimista y reglas de clase.

CODIGO

```
public async Task<AsientosVueloDTO> ObtenerAsientosVuelo(int vueloId, int usuarioId)
```

Retorna el mapa completo de asientos de un vuelo para un usuario especifico. Calcula el layout dinamico por clase, identifica los asientos ocupados por otros pasajeros y lista los boletos del usuario en su reservacion activa.

CODIGO

```
public async Task CambiarAsiento(int boletoId, string nuevoAsiento, int usuarioId)
```

Cambia el asiento de un boleto del usuario autenticado. Verifica que el boleto pertenezca al usuario, que la reservacion este pendiente y no expirada, que el asiento sea valido en el layout del vuelo, que este disponible con bloqueo pesimista y que

corresponda a la clase del boleto.

AvionRepository

Repositorio de aviones. Gestiona el CRUD completo de aviones e imagenes asociadas. Permite consultar la flota disponible para la asignacion de vuelos.

```
CODIGO  
public async Task<List<Avion>> ObtenerTodos()
```

Retorna la lista completa de aviones con su imagen asociada, ordenados por marca y modelo.

```
CODIGO  
public async Task<Avion?> ObtenerPorId(int id)
```

Retorna un avion especifico con su imagen. Retorna null si no existe el ID dado.

```
CODIGO  
public async Task<int> Crear(Avion avion)
```

Inserta un nuevo avion en la base de datos y retorna el ID generado.

```
CODIGO  
public async Task<bool> Actualizar(Avion avion)
```

Actualiza el modelo, marca y capacidad de un avion existente. Retorna true si se modifico al menos una fila.

```
CODIGO  
public async Task<bool> Eliminar(int id)
```

Elimina un avion y su imagen asociada de la base de datos. Retorna true si la eliminacion fue exitosa.

```
CODIGO  
public async Task GuardarImagen(int avionId, string imagenBase64)
```

Guarda o actualiza la imagen en Base64 de un avion. Si ya existe un registro de imagen lo actualiza; si no, lo inserta.

```
CODIGO  
public async Task EliminarImagen(int avionId)
```

Elimina la imagen asociada a un avion segun su ID.

CODIGO

```
public async Task<string?> ObtenerImagen(int avionId)
```

Retorna la imagen en Base64 de un avion. Retorna null si no tiene imagen.

CiudadRepository

Repositorio de ciudades. Permite buscar o crear ciudades asociadas a un pais dentro de la base de datos, reutilizando registros existentes cuando sea posible.

CODIGO

```
public async Task<int> ObtenerOCrearId(string nombre, int paisId, SqlConnection connection, SqlTransaction transaction = null)
```

Busca una ciudad por nombre y pais. Si no existe la crea y retorna su ID. Acepta una conexion y transaccion opcionales para participar en operaciones mayores.

ComentarioRepository

Repositorio de comentarios y reseñas. Gestiona la creación de reseñas de ruta, respuestas a comentarios, consultas por ruta o usuario, y la inclusión del voto del usuario autenticado en los resultados.

CODIGO

```
public async Task<ComentarioDTO> CrearComentarioRuta(int usuarioId, CrearComentarioRutaDTO dto)
```

Crea una reseña con calificación de estrellas para una ruta. Verifica que la ruta exista, que el usuario haya viajado en ella con una reservación completada y que no haya dejado ya una reseña en esa ruta. Retorna el DTO del comentario creado.

CODIGO

```
public async Task<ComentarioDTO> CrearRespuesta(int usuarioId, CrearRespuestaDTO dto)
```

Crea una respuesta a un comentario existente. Verifica que el comentario padre exista y obtiene la ruta asociada para vincular la respuesta. Las respuestas no incluyen calificación de estrellas. Retorna el DTO del comentario creado.

CODIGO

```
public async Task<List<ComentarioDTO>> ObtenerComentariosPorRuta(int rutaId)
```

Retorna todos los comentarios y respuestas de una ruta específica, agrupados para que las respuestas aparezcan junto a su comentario padre, ordenados cronológicamente.

CODIGO

```
public async Task<List<ComentarioConVotoDTO>> ObtenerTodosConVoto(int usuarioId)
```


Retorna todos los comentarios del sistema incluyendo el voto del usuario autenticado en cada comentario. Ordenados por fecha descendente.

CODIGO

```
public async Task<List<ComentarioDTO>> ObtenerComentariosPorUsuario(int usuarioId)
```

Retorna todos los comentarios realizados por un usuario específico, ordenados por fecha descendente. Incluye reseñas y respuestas.

CODIGO

```
public async Task<List<ComentarioConVotoDTO>> ObtenerComentariosRutaConVoto(int rutaId, int usuarioId)
```

Retorna los comentarios de una ruta incluyendo el voto del usuario autenticado en cada comentario. Agrupa respuestas junto a su padre y ordena cronológicamente.

ConfirmarReservacionAgenciaRepository

Repositorio de confirmación de reservaciones para agencias. Gestiona el proceso transaccional de confirmar una reservación pendiente: validaciones, creación de factura y actualización de estados de boletos y reservación.

CODIGO

```
public async Task<ConfirmacionAgenciaDTO> ConfirmarReservacion(
```

Confirma una reservación pendiente de una agencia. Verifica pertenencia a la agencia, estado y expiración de la reservación, que todos los boletos tengan pasajero asignado, crea la factura y actualiza estados de boletos y reservación dentro de una transacción atómica. Retorna el DTO con los datos de la confirmación realizada.

DownRepository

Repositorio de votos (downs) en comentarios. Permite votar a favor o en contra de un comentario, cambiar un voto existente, quitarlo y consultar el voto actual de un usuario en un comentario específico.

CODIGO

```
public async Task<ResultadoVotoDTO> VotarComentario(int usuarioId, VotarComentarioDTO dto)
```

Registra o actualiza el voto de un usuario en un comentario. Si es nuevo crea el registro; si ya voto igual lanza excepción; si cambio de voto actualiza el registro. Actualiza el contador de downs en el comentario. Retorna el resultado con el nuevo conteo y la acción realizada.

CODIGO

```
public async Task<ResultadoVotoDTO> QuitarVoto(int usuarioId, int comentarioId)
```

Elimina el voto de un usuario en un comentario y actualiza el contador de downs. Lanza excepcion si el usuario no habia votado en ese comentario. Retorna el resultado con el nuevo conteo y la accion 'voto_eliminado'.

CODIGO

```
public async Task<int?> ObtenerVotoUsuario(int usuarioId, int comentarioId)
```

Retorna el valor del voto del usuario en el comentario indicado. Retorna null si el usuario no ha votado en ese comentario.

FacturaRepository

Repositorio de facturacion. Gestiona el proceso transaccional de compra de una reservacion por parte de un usuario: validaciones, creacion de factura y actualizacion de estados de boletos y reservacion.

CODIGO

```
public async Task<CompraRealizadaDTO> ComprarReservacion(
```

Procesa el pago de una reservacion pendiente del usuario autenticado. Verifica propiedad, estado y expiracion de la reservacion, que todos los boletos tengan pasajero asignado, crea la factura y actualiza estados de boletos y reservacion dentro de una transaccion atomica. Retorna el DTO con los datos de la compra realizada.

GestionReservacionRepository

Repositorio de gestion de reservaciones para usuarios. Permite consultar el historial de reservaciones, obtener el detalle de una reservacion con sus boletos y pasajeros, cancelar reservaciones y verificar si se puede cancelar segun las reglas de tiempo antes del vuelo.

CODIGO

```
public async Task<List<ReservacionDetalleDTO>> ObtenerReservacionesPorUsuario(int usuarioId)
```

Retorna el historial completo de reservaciones de un usuario con sus boletos asociados. Incluye informacion de vuelo, ruta, avion y datos del pasajero por cada boleto. Ordenadas por fecha de creacion descendente.

CODIGO

```
public async Task<ReservacionDetalleDTO> ObtenerReservacionPorId(int reservacionId, int usuarioId)
```

Retorna el detalle completo de una reservacion especifica del usuario, incluyendo boletos con datos de vuelo, ruta, pasajeros y la factura si existe. Retorna null si la reservacion no existe o no pertenece al usuario.

CODIGO

```
public async Task<ResumenReservacionesDTO> ObtenerResumenReservaciones(int usuarioId)
```

Retorna un resumen estadistico de las reservaciones del usuario: totales por estado (pendiente, confirmada, cancelada, expirada, completada) y el monto total gastado en reservaciones confirmadas y completadas.

CODIGO

```
public async Task CancelarReservacion(int reservacionId, int usuarioId, string motivo)
```

Cancela una reservacion pendiente o confirmada del usuario. Valida que la reservacion pertenezca al usuario, que este en estado cancelable, y para reservaciones confirmadas que falten mas de 24 horas para el vuelo. Libera los boletos, devuelve disponibilidad a los vuelos y actualiza el estado de la reservacion con el motivo de cancelacion.

CODIGO

```
public async Task<int> ObtenerUsuarioWebIdDeAgencia(int agenciaId)
```

Obtiene el ID del usuario webservice asociado a la agencia indicada. Se usa para operaciones de gestion que requieren conocer el propietario de la agencia.

CODIGO

```
public async Task<PuedeCancelarDTO> PuedeCancelar(int reservacionId, int usuarioId)
```

Verifica si una reservacion puede ser cancelada por el usuario. Evalua el estado actual y, para reservaciones confirmadas, si faltan mas de 24 horas para el vuelo. Retorna un DTO con el resultado y el motivo de la decision.

HotelAliadoRepository

Repositorio de hoteles aliados. Gestiona la consulta de hoteles activos para la busqueda dinamica, y la creacion, consulta y administracion de hoteles registrados tanto por usuarios Webservice como por el administrador del sistema. Incluye el guardado del token de sesion resultante del handshake con la aerolinea.

CODIGO

```
public async Task<List<HotelAliadoConexionDTO>> ObtenerHotelesActivos()
```

Retorna todos los hoteles aliados activos con su URL y TokenHASH para que el service pueda consultarlos dinamicamente.

CODIGO

```
public async Task<HotelAliadoConexionDTO> ObtenerHotelActivoPorId(int id)
```

Retorna un hotel aliado activo por su ID. Retorna null si no existe o no esta activo. Se usa para busquedas de disponibilidad donde el estado importa.

- Param id - ID del registro HotelAliado a buscar.

CODIGO

```
public async Task<HotelAliadoConexionDTO?> ObtenerHotelParaHandshake(int id)
```

Retorna los datos basicos de un hotel aliado por su ID sin filtrar por estado. Se usa exclusivamente para el proceso de handshake: el admin debe poder hacer handshake independientemente del estado actual del hotel, siempre que el hotel tenga una URL de API configurada. Retorna null si el ID no existe en la tabla.

- Param id - ID del registro HotelAliado a buscar.

CODIGO

```
public async Task<bool> GuardarTokenHash(int hotelId, string tokenHash)
```

Actualiza el TokenHASH de un hotel aliado con el token de sesion recibido tras completar el handshake. Este token se usa para autenticar las llamadas posteriores de la aerolinea hacia el hotel. Retorna true si la actualizacion fue exitosa.

CODIGO

```
public async Task<bool> UsuarioYaTieneHotelAliado(int usuarioId)
```

Verifica si el usuario webservice dado ya tiene un hotel aliado registrado. Retorna true si existe al menos un hotel para ese usuario.

CODIGO

```
public async Task<bool> UsuarioYaTieneAgencia(int usuarioId)
```

Verifica si el usuario webservice dado ya tiene una agencia de viaje registrada. Se usa para impedir que un usuario tenga tanto un hotel como una agencia. Retorna true si existe al menos una agencia para ese usuario.

CODIGO

```
public async Task<int> ObtenerRolUsuario(int usuarioId)
```

Consulta el RolID del usuario especificado. Retorna 0 si el usuario no existe. Se usa para validar que el usuario a asignar tenga rol Webservice.

CODIGO

```
public async Task<MiHotelDTO> CrearHotelWebservice(int usuarioId, CrearHotelWebserviceDTO dto)
```

Inserta un nuevo hotel aliado vinculado al usuario Webservice indicado. El EstadoID se establece en 1 (activo) por defecto. El TokenHASH se deja vacio ya que se genera automaticamente al establecer la conexion con la aerolinea. Retorna el DTO con los datos del hotel recién creado.

CODIGO

```
public async Task<MiHotelDTO?> ObtenerHotelPorUsuarioId(int usuarioId)
```

Retorna los datos del hotel aliado del usuario webservice indicado. No incluye el TokenHASH ya que es informacion interna del sistema. Retorna null si el usuario no tiene ningun hotel registrado.

CODIGO

```
public async Task<List<HotelAdminDTO>> ObtenerTodosAdmin()
```

Retorna el listado completo de hoteles aliados con los datos del usuario Webservice asignado. No incluye TokenHASH. Destinado al panel de administracion.

CODIGO

```
public async Task<HotelAdminDTO> CrearHotelAdmin(CrearHotelAdminDTO dto)
```

Crea un nuevo hotel aliado desde el panel de administracion, asignandolo directamente al usuario Webservice indicado. El TokenHASH se deja vacio. Retorna el DTO completo del hotel creado con el ID generado.

CODIGO

```
public async Task<bool> ActualizarEstado(int hotelId, int estadoId)
```

Actualiza el EstadoID de un hotel aliado. Retorna true si la actualizacion fue exitosa.

CODIGO

```
public async Task<bool> AsignarUsuario(int hotelId, int usuarioId)
```

Asigna o reasigna un usuario Webservice a un hotel aliado existente. Retorna true si la actualizacion fue exitosa.

CODIGO

```
public async Task<bool> ActualizarUrls(int hotelId, string url, string urlParaUsuario)
```

Actualiza la URL de la API y la URL publica para usuarios de un hotel aliado. Retorna true si la actualizacion fue exitosa.

MetricasRepository

Repositorio de metricas del sistema. Provee datos estadisticos sobre busquedas de vuelos, rutas mas buscadas, distribucion por tipo de acceso (Web/REST), ingresos reales desde facturas y listados paginados para exportacion o analisis.

CODIGO

```
public async Task<List<BusquedasPorDiaDTO>> ObtenerBusquedasPorDia(
```

Retorna el conteo de busquedas agrupadas por dia dentro del rango de fechas indicado. Si no se indica rango se usan los ultimos 30 dias. Ordenado ascendente.

CODIGO

```
public async Task<List<RutaMasBuscadaDTO>> ObtenerRutasMasBuscadas(
```

Retorna las 10 rutas mas buscadas en el rango de fechas indicado. Opcionalmente filtra por tipo de acceso (Web o REST). Incluye codigos y nombres de ciudad de origen y destino.

CODIGO

```
public async Task<List<BusquedasPorTipoDTO>> ObtenerBusquedasPorTipo(
```

Retorna la cantidad de busquedas agrupadas por tipo (Web y REST) en el rango de fechas indicado. Se usa para graficar la proporcion de cada canal.

CODIGO

```
public async Task<(IngresosKpiDTO kpi, List<DistribucionClaseDTO> dist)> ObtenerIngresos(
```

Calcula los ingresos reales del periodo a partir de las facturas emitidas. Retorna un KPI con totales y ticket promedio, y una distribucion de ingresos y boletos por clase (Turista y Ejecutiva). Excluye reservaciones canceladas.

CODIGO

```
public async Task<MetricasResumenDTO> ObtenerResumen(DateTime? desde, DateTime? hasta)
```

Construye el resumen general de metricas del sistema combinando busquedas por dia, rutas mas buscadas, distribucion por tipo y datos de ingresos para el rango de fechas indicado.

CODIGO

```
public async Task<ListadoBusquedasDTO> ObtenerListado(MetricasFiltroDTO filtro)
```

Retorna un listado paginado de busquedas con filtros por fecha, tipo de acceso y usuario. Incluye conteo total para calcular paginas. Se usa para la tabla de detalle del panel de metricas y para exportaciones.

NacionalidadRepository

Repositorio de nacionalidades. Permite consultar el catalogo completo y buscar o crear registros de nacionalidad segun su nombre.

CODIGO

```
public async Task<List<Nacionalidad>> ObtenerTodas()
```

Retorna la lista completa de nacionalidades ordenadas alfabeticamente.

CODIGO

```
public async Task<int> ObtenerOCrearId(string nombre, SqlConnection connection)
```

Busca una nacionalidad por nombre usando la conexion dada. Si no existe la crea y retorna su ID. Se usa al registrar usuarios con nacionalidades nuevas.

PaisRepository

Repositorio de paises. Permite buscar o crear registros de pais por nombre, reutilizando entradas existentes para mantener la integridad referencial.

CODIGO

```
public async Task<int> ObtenerOCrearId(string nombre, SqlConnection connection, SqlTransaction transaction = null)
```

Busca un pais por nombre. Si no existe lo crea y retorna su ID. Acepta una conexion y transaccion opcionales para participar en transacciones externas.

PerfilRepository

Repositorio de perfil de usuario. Permite consultar y actualizar datos personales del usuario autenticado, incluyendo telefono y contraseña.

CODIGO

```
public async Task<PerfilDTO?> ObtenerPerfil(int usuarioId)
```

Retorna el perfil completo del usuario incluyendo nombre, apellido, correo, telefono, pasaporte, fecha de nacimiento, ciudad y pais. Retorna null si el usuario no existe.

CODIGO

```
public async Task<bool> ActualizarTelefono(int usuarioId, string telefono)
```

Actualiza el numero de telefono del usuario indicado. Retorna true si la actualizacion afecto al menos una fila.

CODIGO

```
public async Task<string?> ObtenerHashContraseña(int usuarioId)
```

Retorna el hash de la contraseña almacenada para el usuario indicado. Retorna null si el usuario no existe.

CODIGO

```
public async Task<bool> ActualizarContraseña(int usuarioId, string nuevoHash)
```

Actualiza el hash de contraseña del usuario con el nuevo valor proporcionado. Retorna true si la actualizacion afecto al menos una fila.

ReservacionAgenciaRepository

Repositorio de reservaciones para agencias. Gestiona la creacion de reservaciones con descuento aplicado, la expiracion manual de reservaciones pendientes y la asignacion de pasajeros a boletos de una reservacion existente.

CODIGO

```
public async Task<ReservacionCreadaDTO> CrearReservacion(List<SeleccionVueloDTO> vuelos, decimal descuento, int agenciaId)
```

Crea una reservacion para la agencia indicada aplicando el descuento configurado. Verifica disponibilidad de boletos con UPDLOCK/ROWLOCK, descuenta asientos del vuelo, asigna asientos secuenciales y retorna el DTO con todos los boletos reservados. La transaccion usa nivel Serializable para evitar condiciones de carrera.

CODIGO

```
public async Task ExpirarReservacion(int reservacionId)
```

Marca como expirada una reservacion pendiente de la agencia. Libera los boletos reservados devolviendo disponibilidad al vuelo correspondiente y cambia el estado de la reservacion a expirado (4). Lanza excepcion si la reservacion no esta pendiente.

CODIGO

```
public async Task<bool> PerteneceAAgenciaYEstaPendiente(int reservacionId, int agenciaId)
```

Verifica si una reservacion pertenece a la agencia indicada y se encuentra en estado pendiente (1). Retorna true si se cumplen ambas condiciones.

CODIGO

```
public async Task AgregarPasajerosAReservacion(int reservacionId, List<DatosPasajeroDTO> pasajeros, int agenciaId)
```

Asigna o actualiza los datos de pasajero para cada boleto de una reservacion de agencia. Verifica que la reservacion pertenezca a la agencia, este pendiente y no haya expirado. Crea o actualiza registros en DatosPasajero y los vincula al boleto correspondiente.

ReservacionRepository

Repositorio de reservaciones para usuarios. Gestiona la creacion de reservaciones con asignacion automatica de asientos, la liberacion de reservaciones expiradas, la asignacion de pasajeros a boletos, la confirmacion y el completado automatico de reservaciones cuyos vuelos ya aterrizaron.

CODIGO

```
public async Task<ReservacionCreadaDTO> CrearReservacion(int? usuarioId, List<SeleccionVueloDTO> vuelos)
```

Crea una nueva reservacion para el usuario indicado. Si el usuario ya tiene reservaciones pendientes, las expira liberando sus asientos antes de continuar. Verifica disponibilidad con UPDLOCK/ROWLOCK, descuenta boletos del vuelo, asigna asientos secuenciales y retorna el DTO con todos los boletos reservados. La transaccion usa nivel Serializable para evitar condiciones de carrera.

CODIGO

```
public async Task<int> LiberarReservasExpiradas()
```

Busca todas las reservaciones en estado pendiente cuya fecha de expiracion ya paso, libera sus boletos devolviendo disponibilidad a los vuelos y cambia el estado de cada reservacion a expirado (4). Retorna la cantidad de reservaciones procesadas.

CODIGO

```
public async Task AgregarPasajerosAReservacion(int reservacionId, List<DatosPasajeroDTO> pasajeros)
```

Asigna o actualiza los datos de pasajero para cada boleto de una reservacion de usuario. Verifica que la reservacion exista, este pendiente y no haya expirado. Crea o actualiza registros en DatosPasajero y los vincula al boleto correspondiente.

CODIGO

```
public async Task ConfirmarReservacion(int reservacionId)
```

Confirma una reservacion pendiente cambiando el estado de todos sus boletos a Vendido (3) y el estado de la reservacion a Confirmada (2). Verifica que todos los boletos tengan pasajero asignado antes de confirmar.

CODIGO

```
public async Task<int> CompletarReservaciones()
```

Busca reservaciones confirmadas (estado 2) cuyos vuelos ya aterrizaron y las marca como completadas (estado 5). Solo completa reservaciones donde todos los vuelos asociados tienen estado 3 (finalizado). Retorna la cantidad de reservaciones completadas.

RutaAgenciaRepository

Repositorio de rutas para agencias. Proporciona acceso de solo lectura al catalogo completo de rutas disponibles con informacion de ciudad, pais y duracion estimada de cada trayecto.

CODIGO

```
public async Task<List<RutaAgenciaDTO>> ObtenerTodasLasRutas()
```

Retorna la lista completa de rutas con ciudad y pais de origen y destino obtenidos a traves de los aeropuertos asociados a cada ruta.

RutaRepository

Repositorio de rutas. Permite listar, crear, buscar y actualizar rutas entre aeropuertos. Incluye introspeccion de esquema para soportar instalaciones con o sin la tabla ZonaHoraria, adaptando las consultas segun la estructura disponible.

CODIGO

```
public async Task<List<RutaDTO>> ObtenerTodas()
```

Retorna todas las rutas con informacion de aeropuertos y zonas horarias. Si la tabla ZonaHoraria o la columna ZonaHorariaID no existen en el esquema actual, omite los JOINS correspondientes y retorna null en esos campos.

CODIGO

```
public async Task<bool> ActualizarDuracion(int rutaId, int minutos)
```

Actualiza la duracion estimada en minutos de una ruta existente. Retorna true si la actualizacion afecto al menos una fila.

CODIGO

```
public async Task<(int duracion, string? tzOrigen, string? tzDestino)> ObtenerInfoRuta(
```

Obtiene la duracion estimada y las zonas horarias de origen y destino para calcular la hora de llegada de un vuelo dado el par de aeropuertos. Si la tabla ZonaHoraria no existe en el esquema, retorna null en las zonas horarias. Retorna (120, null, null) como fallback si la ruta no existe.

```
CODIGO
public async Task<bool> ExisteRuta(int origenId, int destinoId)
```

Verifica si existe una ruta directa entre los aeropuertos de origen y destino indicados. Retorna true si existe al menos una ruta con ese par exacto origen-destino.

```
CODIGO
public async Task<int> CrearRuta(int origenId, int destinoId, int duracionEstimada = 120)
```

Crea una nueva ruta entre los aeropuertos indicados con la duracion estimada dada. Si ya existe una ruta con el mismo par origen-destino, retorna su ID sin duplicar.

TripulacionRepository

Repositorio de tripulacion. Gestiona el CRUD completo de miembros de tripulacion y sus imagenes almacenadas directamente en la tabla MiembroTripulacion. Tambien permite consultar los roles de tripulacion disponibles.

```
CODIGO
public async Task<List<Tripulante>> ObtenerTodos()
```

Retorna la lista de todos los miembros de tripulacion ordenados por ID, incluyendo su imagen en Base64 si esta disponible.

```
CODIGO
public async Task<Tripulante?> ObtenerPorId(int id)
```

Retorna el miembro de tripulacion con el ID indicado. Retorna null si no existe ningun tripulante con ese ID.

```
CODIGO
public async Task<string?> ObtenerNombreRol(int rolId)
```

Retorna el nombre del cargo asociado al rol de tripulacion indicado. Retorna null si el rol no existe.

```
CODIGO
public async Task<int> Crear(Tripulante tripulante)
```

Inserta un nuevo miembro de tripulacion en la base de datos incluyendo su imagen en Base64 si se proporciona. Retorna el ID generado.

CODIGO

```
public async Task<bool> Actualizar(Tripulante tripulante)
```

Actualiza los datos de un miembro de tripulacion existente. Si se proporciona una nueva imagen la incluye en la actualizacion; de lo contrario conserva la imagen anterior sin modificarla. Retorna true si se actualizo al menos una fila.

CODIGO

```
public async Task<bool> Eliminar(int id)
```

Elimina el miembro de tripulacion con el ID indicado. Retorna true si se elimino al menos una fila.

CODIGO

```
public async Task GuardarImagen(int tripulanteId, string imagenBase64)
```

Guarda o reemplaza la imagen en Base64 del tripulante indicado actualizando directamente la columna Imagen de MiembroTripulacion.

CODIGO

```
public async Task EliminarImagen(int tripulanteId)
```

Elimina la imagen del tripulante indicado estableciendo NULL en la columna Imagen.

CODIGO

```
public async Task<List<RolTripulacion>> ObtenerRoles()
```

Retorna la lista de todos los roles de tripulacion disponibles ordenados por ID.

UsuarioRepository

Repositorio de usuarios. Gestiona la creacion, consulta y actualizacion de usuarios, incluyendo la asignacion de nacionalidades, verificacion de duplicados en registro, busqueda por credenciales para autenticacion y administracion de roles.

CODIGO

```
public async Task<int> CrearUsuario(Usuario usuario)
```

Inserta un nuevo usuario en la base de datos y retorna el ID generado. Los campos opcionales como telefono, fecha de nacimiento y ciudad aceptan null o valor por defecto y se almacenan como DBNull en la base de datos.

CODIGO

```
public async Task AgregarNacionalidades(int usuarioId, List<string> nacionalidades)
```

Asocia una lista de nacionalidades a un usuario recién creado. Por cada nacionalidad obtiene o crea el registro en la tabla Nacionalidad y luego inserta el vínculo en UsuarioNacionalidad.

CODIGO

```
public async Task<RegisterConstraint> VerificarExistencia(string correo, string username, string pasaporte)
```

Verifica si ya existe un usuario con el correo, username o pasaporte indicados. Retorna un objeto RegisterConstraint con las flags correspondientes a cada campo que ya este en uso, para informar al cliente que datos causan conflicto.

CODIGO

```
public async Task<Usuario?> ObtenerPorCorreoOUsername(string correoOUsername)
```

Busca un usuario por correo electronico o username para el proceso de autenticacion. Retorna el objeto Usuario completo con su hash de contrasena y rol, o null si no existe ningun usuario con esas credenciales.

CODIGO

```
public async Task<string?> ObtenerNombreRol(int rolId)
```

Retorna el nombre del rol asociado al ID indicado. Retorna null si el rol no existe en la tabla Rol.

CODIGO

```
public async Task<bool> ActualizarRol(int usuarioId, int nuevoRolId)
```

Actualiza el rol de un usuario al nuevo rol indicado. Retorna true si la actualizacion afecto al menos una fila.

CODIGO

```
public async Task<bool> UsuarioExiste(int usuarioId)
```

Verifica si existe un usuario con el ID indicado. Retorna true si se encuentra al menos un registro con ese ID.

CODIGO

```
public async Task<bool> RolExiste(int rolId)
```

Verifica si existe un rol con el ID indicado en la tabla Rol. Retorna true si se encuentra al menos un registro con ese ID.

CODIGO

```
public async Task<List<Usuario>> ObtenerTodos()
```

Retorna la lista completa de usuarios registrados en el sistema, ordenados por ID. Incluye todos los campos del perfil pero no incluye las nacionalidades asociadas.

VueloAdminInternoRepository

Repositorio interno de actualizacion de estados de vuelos. Ejecuta el proceso automatico que transiciona vuelos de 'A tiempo' (1) a 'En transcurso' (2) o 'Finalizado' (3) segun la hora actual en relacion con sus horas de salida y llegada.

CODIGO

```
public async Task<(int enTranscurso, int finalizados)> ActualizarEstadosVuelos()
```

Actualiza el estado de todos los vuelos cuya hora de salida o llegada ya ocurrio. Pasa a estado 2 (en transcurso) los vuelos que ya salieron pero no aterrizaron, y a estado 3 (finalizado) los vuelos cuya hora de llegada ya paso. Retorna una tupla con la cantidad de vuelos pasados a cada estado.

VueloRepository

Repositorio principal de vuelos. Soporta busqueda por termino libre, busqueda directa por ruta y fecha, y busqueda con escalas mediante BFS por capas. Tambien registra cada busqueda realizada para alimentar las metricas del sistema.

CODIGO

```
public async Task<List<VueloDetalleDTO>> BusquedaGeneral(string query)
```

Realiza una busqueda libre de vuelos activos que coincidan con el termino ingresado contra ciudad, pais, aeropuerto o numero de vuelo. Retorna hasta 50 resultados futuros con disponibilidad, ordenados por fecha y hora de salida.

CODIGO

```
public async Task<List<VueloDetalleDTO>> BuscarVuelos(
```

Busca vuelos directos entre dos aeropuertos en una fecha especifica con la cantidad de pasajeros indicada. Permite filtrar por clase (Turista o Ejecutivo). Retorna los vuelos disponibles ordenados por hora de salida, cada uno con su lista de tripulantes asignados.

CODIGO

```
public async Task<List<VueloConEscalaDTO>> BuscarVuelosConEscalas(
```

Busca combinaciones de vuelos con escalas entre dos aeropuertos usando BFS por capas. Aplica reglas de escala entre 1h y 12h, limita la duracion total de vuelo al 1.5x de la ruta directa y evita ciclos en aeropuertos intermedios. Admite hasta maxEscalas (default 3) y filtra por clase y disponibilidad. Retorna los resultados ordenados por duracion total incluyendo tiempo de espera.

CODIGO

```
public async Task GuardarBusqueda(
```

Registra una busqueda de vuelos en la tabla Busqueda para uso en metricas. Solo inserta el registro si existe una ruta directa entre los aeropuertos indicados.

ESTRUCTURA

Models**Aeropuerto**

Entidad que representa un aeropuerto registrado en el sistema. Contiene codigo IATA, nombre, ciudad asociada y zona horaria opcional.

Avion

Entidad que representa una aeronave registrada en la flota de la aerolinea. Almacena marca, modelo, capacidad de pasajeros e imagen opcional en Base64.

Boleto

Entidad que representa un boleto de avion asignado a un vuelo especifico. Contiene numero de boleto, asiento, precio, clase, estado y referencias a la reservacion y datos del pasajero.

BusquedaVuelo

Entidad que registra una busqueda de vuelo realizada por un usuario o agencia. Almacena origen, destino, fechas, cantidad de pasajeros y tipo de viaje para fines de analisis y metricas del sistema.

Ciudad

Entidad que representa una ciudad del catalogo geografico del sistema. Pertenece a un pais y sirve como referencia para aeropuertos y usuarios.

Comentario

Entidad que representa un comentario o resena publicado por un usuario sobre una ruta. Incluye calificacion por estrellas, contenido textual, conteo de votos negativos y fecha.

DatosPasajero

Entidad que almacena la informacion personal de un pasajero asociado a un boleto. Incluye nombre, apellido, numero de pasaporte, telefono y ubicacion geografica.

Down

Entidad que registra el voto de un usuario sobre un comentario. El campo Valor indica el tipo de voto: 1 para positivo y -1 para negativo. Garantiza que cada usuario solo pueda votar una vez por comentario.

Guardarbusqueda

DTO utilizado para persistir los parametros de una busqueda de vuelo realizada por el usuario. Contiene origen, destino, fechas, numero de pasajeros y tipo de viaje.

Loginresponse

DTO de respuesta devuelto al cliente despues de un inicio de sesion exitoso. Contiene identificacion del usuario, nombre, correo y datos del rol asignado.

Nacionalidad

Entidad que representa una nacionalidad disponible en el catalogo del sistema. Se utiliza para asociar una o mas nacionalidades a un usuario registrado.

Pais

Entidad que representa un pais del catalogo geografico del sistema. Sirve como referencia para ciudades, aeropuertos y datos de usuario.

RegisterConstraint

DTO que indica que campos unicos ya existen en el sistema durante el proceso de registro. Se utiliza para validar disponibilidad de correo, username y pasaporte antes de crear el usuario.

Reservacion

Entidad que representa una reservacion de vuelo realizada por un usuario. Contiene numero unico de reserva, fechas de creacion y expiracion, total a pagar y estado actual del proceso de reserva.

RolTripulacion

Entidad que define un rol o cargo dentro de la tripulacion de un vuelo. Ejemplos de cargos: Piloto, Copiloto, Auxiliar de vuelo.

Tripulante

Entidad que representa a un miembro de la tripulacion registrado en el sistema. Contiene nombre, apellido, rol asignado e imagen opcional en Base64.

Usuario

Entidad principal que representa a un usuario registrado en el sistema. Almacena credenciales de acceso, datos personales, ubicacion geografica y el rol asignado que determina los permisos dentro de la plataforma.

UsuarioNacionalidad

Entidad de relacion que vincula a un usuario con una o mas nacionalidades. Permite que un usuario tenga multiples nacionalidades registradas en el sistema.

Vuelo

Entidad que representa un vuelo programado en el sistema. Contiene numero de vuelo, fecha, horarios de salida y llegada, avion asignado, ruta, disponibilidad de boletos por clase y precios de cada clase.

DTOs

AeropuertoDTO

DTO de lectura que expone la informacion publica de un aeropuerto. Incluye codigo IATA, ciudad, pais, imagen opcional y zona horaria configurada.

AgregarPasajerosDTO

DTO para asignar los datos de los pasajeros a los boletos de una reservacion existente. Contiene el identificador de la reservacion y la lista de pasajeros con sus datos personales.

AsientosVueloAgenciaDTO

DTO de respuesta que muestra el mapa de asientos de un vuelo para una agencia. Incluye configuracion del avion, asientos ocupados y boletos reservados por la agencia.

AvionDTO

DTO de lectura que expone la informacion de una aeronave registrada en la flota. Incluye marca, modelo, capacidad, nombre completo e imagen opcional en Base64.

BuscarVueloAgenciaDTO

DTO de peticion para que una agencia busque vuelos disponibles. Permite filtrar por origen, destino, fecha, cantidad de pasajeros, clase y rango de precios.

BuscarVueloDTO

DTO de peticion para buscar vuelos disponibles desde el portal web. Permite filtrar por aeropuerto de origen y destino, fecha, cantidad de pasajeros, clase y rango de precios.

CambiarAsientoRequestDTO

DTO de peticion para cambiar el asiento asignado a un boleto de la reservacion activa. Contiene unicamente el nuevo asiento solicitado por el usuario.

CambiarRolDTO

DTO de peticion para cambiar el rol de un usuario desde el panel de administracion. Contiene el identificador del usuario y el identificador del nuevo rol a asignar.

CancelarReservacionDTO

DTO de peticion para cancelar una reservacion activa. El campo Motivo es opcional y puede enviarse como null.

CiudadDTO

DTO de lectura que expone la informacion de una ciudad junto con su pais de pertenencia. Incluye nombre completo con formato "Ciudad, Pais" para uso en listas desplegables.

ComprarReservacionDTO

DTO de peticion para confirmar la compra de una reservacion pendiente. Contiene datos de facturacion y datos de tarjeta para validacion de formato. Los datos de la tarjeta nunca se persisten en el sistema.

ConfirmacionAgenciaDTO

DTO de peticion para que una agencia confirme y pague una reservacion pendiente. Contiene los datos de facturacion requeridos para generar la factura.

CrearAeropuerto

DTO para registrar o actualizar un aeropuerto desde el panel de administracion. Requiere nombre, codigo IATA, ciudad y pais. Acepta imagen en Base64 y zona horaria IANA de forma opcional.

CrearAgenciaDTO

DTO para que un administrador cree una nueva agencia y la asigne a un usuario Webservice. Incluye nombre, correo, ID del usuario web, porcentaje de descuento y URL publica.

CrearAvionDTO

DTO para registrar o actualizar una aeronave en el sistema. Requiere marca, modelo y capacidad de pasajeros. Acepta imagen en Base64 de forma opcional.

CrearComentarioDTO

DTO de peticion para publicar un nuevo comentario o resena sobre una ruta especifica. Contiene identificador de la ruta, calificacion por estrellas y texto del comentario.

CrearReservacionDTO

DTO de peticion para crear una nueva reservacion de vuelo. El `UsuarioId` se obtiene de la cookie de sesion en el controller. Contiene la lista de vuelos seleccionados con clase y cantidad de pasajeros.

CrearTripulanteDTO

DTO para registrar o actualizar un miembro de la tripulacion en el sistema. Requiere nombre, apellido y rol asignado. Acepta imagen en Base64 de forma opcional.

CrearUsuarioDTO

DTO de peticion para registrar un nuevo usuario en el sistema. Contiene credenciales de acceso, datos personales, ubicacion geografica y lista de nacionalidades. El rol por defecto es usuario regular (`RoleId = 2`).

CrearVueloAdminDTO

DTO para la creacion de un nuevo vuelo desde el panel de administracion. Contiene numero de vuelo, aeropuertos, avion, fecha, hora de salida local, disponibilidad y precios por clase, y los tripulantes asignados.

FacturaDTO

DTO de lectura que expone los datos de una factura generada al confirmar una reservacion. Incluye identificador, fecha, datos fiscales y monto total de la transaccion.

HandshakeRequestDTO

DTO de peticion para iniciar el proceso de autenticacion por handshake con una agencia externa. Contiene el token de entrada proporcionado por la agencia y su URL de origen.

HotelesAliados

DTO con la informacion completa de un hotel disponible retornado al frontend. Incluye los datos del hotel de la API externa y la referencia al aliado registrado en la base de datos de aerolineas.

LoginRequestDto

DTO de peticion para autenticar a un usuario en el sistema. Acepta correo electronico o username junto con la contrasena del usuario.

MetricasDTO

DTO que agrupa la cantidad total de búsquedas realizadas en un día específico. Utilizado en gráficas y reportes del panel de métricas administrativo.

NacionalidadDto

DTO de lectura que expone el identificador y nombre de una nacionalidad del catálogo. Utilizado en listas desplegables durante el registro de usuarios.

PerfilDTO

DTO de lectura con la información del perfil del usuario autenticado. Incluye datos personales, credenciales de contacto y ubicación geográfica.

ReservacionCreadaDTO

DTO de respuesta devuelto al crear una nueva reservación exitosamente. Incluye identificador, número de reservación, expiración, total y lista de boletos asignados.

ReservacionDetalleDTO

DTO de lectura con el detalle completo de una reservación. Incluye estado, usuario propietario, factura asociada, lista de boletos y datos de cancelación si aplica.

ResultadoBusquedaDTO

DTO de respuesta que agrupa los resultados de una búsqueda de vuelos. Separa los vuelos directos disponibles de los itinerarios con escala.

RutaAgenciaDTO

DTO de lectura que expone la información de una ruta disponible para agencias. Incluye ciudad y país de origen y destino, además de la duración estimada en minutos.

RutaDTO

DTO para mostrar una ruta en el panel de administración. Incluye aeropuertos, zonas horarias y duración editable.

SubirImagenDTO

DTO para subir una imagen codificada en Base64 de forma independiente a cualquier entidad. Se utiliza en endpoints de carga de imágenes para aeropuertos, aviones y tripulantes.

TokenHotelDTO

DTO con los datos necesarios para solicitar un token de alianza a un hotel.

TripulanteDTO

DTO de lectura que expone la informacion de un tripulante registrado en el sistema. Incluye nombre completo, rol de tripulacion e imagen opcional en Base64.

UsuarioInfoDTO

DTO de lectura con la informacion completa de un usuario para uso administrativo. Incluye datos personales, ubicacion, rol asignado y lista de nacionalidades registradas.

VotarComentarioDTO

DTO de peticion para registrar o actualizar el voto de un usuario sobre un comentario. El campo Valor debe ser 1 para voto positivo o -1 para voto negativo.

VueloDetalleDTO

DTO de lectura con el detalle completo de un vuelo para mostrar en resultados de busqueda y paneles de administracion. Incluye estado, avion, aeropuertos de origen y destino, precios, disponibilidad por clase y lista de tripulantes asignados.

VueloHistorialDTO

DTO de lectura que expone el historial de un vuelo para reportes administrativos. Incluye numero de vuelo, ruta, horarios, estado, capacidad, boletos vendidos y precios.

Helpers

AgenciaAuthMiddleware

Representa la identidad de una agencia externa autenticada. Contiene los datos basicos que se propagan en el contexto HTTP tras validar el token.

CODIGO

```
public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
```

Ejecuta la validacion del token de agencia antes de que el action del controller sea invocado. Extrae el token del header X-Agencia-Token, consulta la base de datos para verificarlo y almacena los datos de la agencia en HttpContext.Items si es valido. Retorna 401 si el token esta ausente o no corresponde a ninguna agencia registrada.

CustomAssemblyLoadContext

Contexto de carga de ensamblados personalizado utilizado para cargar librerias nativas en tiempo de ejecucion de forma aislada, principalmente para la generacion de PDFs. Hereda de AssemblyLoadContext para permitir la carga de DLLs no administradas por ruta absoluta.

CODIGO

```
public IntPtr LoadUnmanagedLibrary(string absolutePath)
```

Carga una libreria nativa no administrada desde la ruta absoluta indicada y retorna un handle a la misma.

EmailHelper

Clase estatica de utilidad para el envio de correos electronicos mediante SMTP. Encapsula la configuracion del servidor de correo y expone metodos para enviar mensajes HTML simples, con copia oculta, y para escapar texto en HTML.

CODIGO

```
public static async Task Enviar(string destinatario, string asunto, string cuerpoHtml)
```

Envia un correo electronico con cuerpo HTML al destinatario especificado. Utiliza el servidor SMTP configurado con SSL en el puerto 587.

CODIGO

```
public static async Task EnviarConCopia(string destinatario, string asunto, string cuerpoHtml, string copiaOculta)
```

Envia un correo electronico con cuerpo HTML al destinatario e incluye una copia oculta (BCC) a la direccion indicada, si esta no esta vacia.

CODIGO

```
public static string Esc(string texto)
```

Escapa los caracteres especiales de HTML en el texto recibido para evitar inyeccion de etiquetas al incrustar contenido dinamico en plantillas HTML. Retorna cadena vacia si el texto es nulo o vacio.

EmailTemplates

Clase estatica que centraliza todas las plantillas HTML de correos electronicos enviados por la aplicacion. Cada metodo genera y retorna el HTML completo listo para ser usado como cuerpo del mensaje en EmailHelper.

CODIGO

```
public static string CorreoBienvenida()
```

Genera el HTML del correo de bienvenida que se envia al usuario inmediatamente despues de crear su cuenta en el sistema. Incluye los datos personales registrados, credenciales de acceso y una nota de seguridad sobre el manejo de la contrasena.

CODIGO

```
public static string CorreoReservacion(ReservacionDetalleDTO reservacion)
```

Genera el HTML del correo de detalle de reservacion que se envia al usuario con el comprobante completo. Incluye la tabla de boletos, datos de pasajeros si los hay, el total de la reservacion y los terminos y condiciones del viaje.

CODIGO

```
public static string CorreoContacto(string nombre, string correo, string asunto, string mensaje)
```

Genera el HTML del correo de notificacion que se envia al administrador cuando un usuario envia un mensaje a traves del formulario de contacto. Incluye el nombre, correo, asunto y cuerpo del mensaje del remitente, junto con un boton para responder directamente.

CODIGO

```
public static string CorreoNewsletter(string correo)
```

Genera el HTML del correo de notificacion al administrador cuando un nuevo usuario se suscribe al boletin informativo de Broom AirLine. Muestra el correo del suscriptor y un boton para contactarlo directamente.

CODIGO

```
public static string CorreoConfirmacion(string nombreUsuario, string noReservacion, decimal total)
```

Genera el HTML del correo de confirmacion de reservacion que se envia al usuario una vez que el pago y la confirmacion han sido procesados exitosamente. Muestra el numero de reservacion y el total cobrado.

CODIGO

```
public static string CorreoCancelacion(string nombreUsuario, string noReservacion)
```

Genera el HTML del correo de aviso de cancelacion que se envia al usuario cuando su reservacion ha sido cancelada, ya sea por el propio usuario o por el sistema. Indica el numero de reservacion afectado y datos de contacto.

PasswordHasher

Clase estatica de utilidad para el manejo seguro de contraseñas mediante BCrypt. Provee metodos para generar el hash de una contraseña en texto plano y para verificar si una contraseña ingresada coincide con su hash almacenado.

CODIGO

```
public static string Hash(string password)
```

Genera un hash seguro de la contraseña proporcionada usando el algoritmo BCrypt. El hash resultante incluye el salt embebido y es apto para almacenarse en la base de datos.

CODIGO

```
public static bool Verify(string password, string hash)
```

Verifica si la contraseña en texto plano coincide con el hash previamente almacenado. Retorna true si la contraseña es correcta, false en caso contrario.

PdfHtmlHelper

Clase estatica que genera el HTML utilizado para producir el comprobante de reservacion en formato PDF. Construye un documento HTML completo con el detalle de boletos, datos del pasajero, informacion fiscal si aplica, subtotales por vuelo y los terminos y condiciones del servicio.

CODIGO

```
public static string GenerarComprobante(ReservacionDetalleDTO reservacion)
```

Genera el HTML completo del comprobante de reservacion optimizado para impresion en formato A5 horizontal. Incluye encabezado con numero y estado de la reservacion, tabla de boletos con subtotales por vuelo, seccion de datos de pasajeros, datos fiscales si la reservacion tiene factura asociada, y pie de pagina institucional.

SessionHelper

Clase estatica de utilidad para acceder a los datos del usuario autenticado almacenados en los claims del JWT. Centraliza los nombres de los claims y expone metodos para leer el ID, rol, nombre y correo del usuario activo sin necesidad de manipular strings de claims directamente en los controllers.

CODIGO

```
public static int? GetUsuarioId(HttpContext context)
```

Retorna el ID del usuario autenticado extraido del claim JWT. Retorna null si el usuario no tiene sesion activa o el claim no existe.

CODIGO

```
public static int? GetRolId(HttpContext context)
```

Retorna el ID del rol del usuario autenticado extraido del claim JWT. Retorna null si el usuario no tiene sesion activa o el claim no existe.

CODIGO

```
public static string? GetRolNombre(HttpContext context)
```

Retorna el nombre del rol del usuario autenticado segun el claim de rol estandar de ASP.NET. Retorna null si no hay sesion activa.

CODIGO

```
public static string? GetNombre(HttpContext context)
```

Retorna el nombre completo del usuario autenticado segun el claim de nombre estandar. Retorna null si no hay sesion activa.

CODIGO

```
public static string? GetCorreo(HttpContext context)
```

Retorna el correo electronico del usuario autenticado segun el claim de email estandar. Retorna null si no hay sesion activa.

CODIGO

```
public static bool EstaAutenticado(HttpContext context)
```

Indica si el contexto HTTP actual corresponde a un usuario con sesion autenticada. Retorna true si el usuario tiene una identidad valida y autenticada.

CODIGO

```
public static bool TieneRol(HttpContext context, string rolNombre)
```

Verifica si el usuario autenticado posee el rol indicado segun los claims del JWT. Retorna true si el usuario pertenece al rol especificado.

TarjetaHelper

Clase estatica de utilidad para la validacion y deteccion de tarjetas de credito o debito. Realiza comprobaciones de formato superficiales sobre los datos de la tarjeta sin realizar ningun cargo ni verificacion con entidades bancarias externas.

CODIGO

```
public static void ValidarFormato(
```

Valida el formato de los datos de una tarjeta de pago: numero de 16 digitos, nombre del titular con caracteres validos, fecha de expiracion en formato MM/YY no vencida, y CVV de 3 o 4 digitos. Lanza una excepcion con mensaje descriptivo si alguno de los campos no cumple el formato esperado. No verifica si la tarjeta es real ni realiza cargos.

CODIGO

```
public static string DetectarTipo(string numeroTarjeta)
```

Detecta el tipo de red de la tarjeta (Visa, Mastercard, American Express, Discover) a partir del prefijo del numero de tarjeta. Solo tiene caracter informativo y no garantiza que la tarjeta pertenezca realmente a esa red.

TokenHelper

Clase estatica de utilidad para la generacion de tokens seguros y unicos. Utiliza el generador de numeros aleatorios criptograficos del sistema para producir tokens con alta entropia aptos para autenticacion de agencias, restablecimiento de contrasena u otros flujos que requieran tokens opacos.

CODIGO

```
public static string GenerarTokenHash()
```

Genera un token unico y seguro de 64 caracteres hexadecimales en minusculas. Internamente obtiene 32 bytes aleatorios criptograficamente seguros, les aplica SHA-256 y convierte el resultado a cadena hexadecimal.

INFRAESTRUCTURA

Data

DbConnectionFactory

Fabrica de conexiones a la base de datos. Lee la cadena de conexion desde la configuracion de la aplicacion y expone un metodo para obtener nuevas instancias de SqlConnection listas para usar. Se registra como Singleton en el contenedor de dependencias para ser inyectada en los repositorios que acceden a datos con Dapper.

CODIGO

```
public SqlConnection CreateConnection()
```

Crea y retorna una nueva instancia de SqlConnection configurada con la cadena de conexion de la aplicacion. La conexion no se abre automaticamente; el llamador es responsable de abrirla y cerrarla usando un bloque using.

Program

Program.cs

Punto de entrada de la aplicacion. Configura el servidor, registra middleware, controllers y servicios.

```
CODIGO  
var builder = WebApplication.CreateBuilder(args);
```

Inicializa el WebApplication builder con la configuracion del host.

```
CODIGO  
builder.Services.AddControllers();
```

Registra los controllers de la API para que ASP.NET los descubra automaticamente.

```
CODIGO  
builder.Services.AddSingleton<DbConnectionFactory>();
```

Conexion a SQL Server como Singleton: una sola instancia para toda la aplicacion.

```
CODIGO  
builder.Services.AddScoped<PaisRepository>();
```

Repositorios geograficos - Paises: consulta y creacion.

```
CODIGO  
builder.Services.AddScoped<CiudadRepository>();
```

Repositorios geograficos - Ciudades vinculadas a un pais.

```
CODIGO  
builder.Services.AddScoped<NacionalidadRepository>();
```

Repositorios geograficos - Nacionalidades para asignacion a usuarios.

```
CODIGO  
builder.Services.AddScoped<UsuarioRepository>();
```

Repositorios de usuarios - Registro, consulta y validacion de credenciales.

```
CODIGO  
builder.Services.AddScoped<AeropuertoRepository>();
```

Repositorios de vuelos - Aeropuertos con codigos IATA.

```
CODIGO  
builder.Services.AddScoped<VueloRepository>();
```

Repositorios de vuelos - Busqueda, disponibilidad e itinerarios.

```
CODIGO  
builder.Services.AddScoped<ReservacionRepository>();
```

Repositorios de reservaciones - Creacion, consulta y cancelacion.

```
CODIGO  
builder.Services.AddScoped<ComentarioRepository>();
```

Repositorios sociales - Comentarios de usuarios sobre vuelos.

```
CODIGO  
builder.Services.AddScoped<DownRepository>();
```

Repositorios sociales - Votos positivos y negativos en comentarios.

```
CODIGO  
builder.Services.AddScoped<AvionRepository>();
```

Repositorios de flota - Aviones disponibles y capacidad.

```
CODIGO  
builder.Services.AddScoped<TripulacionRepository>();
```

Repositorios de flota - Tripulacion asignada a cada vuelo.

```
CODIGO  
builder.Services.AddScoped<GestionReservacionRepository>();
```

Repositorios administrativos - Gestion de reservaciones desde el panel admin.

```
CODIGO  
builder.Services.AddScoped<AdminVueloRepository>();
```

Repositorios administrativos - Administracion de vuelos desde el panel admin.

```
CODIGO  
builder.Services.AddScoped<VueloAdminInternoRepository>();
```

Repositorios administrativos - Operaciones administrativas avanzadas de vuelos.

```
CODIGO  
builder.Services.AddScoped<PerfilRepository>();
```

Repositorios de usuarios - Datos personales y preferencias del usuario.

```
CODIGO  
builder.Services.AddScoped<FacturaRepository>();
```

Repositorios de pagos - Facturas generadas por reservaciones confirmadas.

```
CODIGO  
builder.Services.AddScoped<MetricasRepository>();
```

Repositorios administrativos - Estadísticas de vuelos, usuarios y reservaciones.

```
CODIGO  
builder.Services.AddScoped<RutaRepository>();
```

Repositorios de vuelos - Rutas aereas entre aeropuertos.

```
CODIGO  
builder.Services.AddScoped<AsientoRepository>();
```

Repositorios de vuelos - Disponibilidad y asignacion de asientos.

```
CODIGO  
builder.Services.AddScoped<AgenciaRepository>();
```

Repositorios de agencias - Agencias registradas en el sistema.

```
CODIGO  
builder.Services.AddScoped<RutaAgenciaRepository>();
```

Repositorios de agencias - Rutas disponibles para agencias.

```
CODIGO  
builder.Services.AddScoped<HotelAliadoRepository>();
```

Repositorios de hotel - Hoteles aliados registrados en el sistema.

```
CODIGO  
builder.Services.AddScoped<TokenHotelService>();
```

Servicio de hotel - Obtener token de descuentos al redireccionar.

```
CODIGO  
builder.Services.AddHealthChecks();
```

Health check para monitoreo del estado del servidor.

```
CODIGO  
builder.Services.AddScoped<AgenciaAuthMiddleware>();
```

Middleware de autenticacion para validar tokens de agencias externas.

```
CODIGO  
builder.Services.AddScoped<ReservacionAgenciaRepository>();
```

Repositorios de agencias - Reservaciones creadas por agencias.

```
CODIGO  
builder.Services.AddScoped<AsientoAgenciaRepository>();
```

Repositorios de agencias - Asientos para operaciones de agencias.

```
CODIGO  
builder.Services.AddScoped<ConfirmarReservacionAgenciaRepository>();
```

Repositorios de agencias - Confirmacion de reservaciones de agencias externas.

```
CODIGO  
builder.Services.AddScoped<NacionalidadService>();
```

Servicios de usuarios - Consulta y asignacion de nacionalidades.

```
CODIGO  
builder.Services.AddScoped<UsuarioService>();
```

Servicios de usuarios - Registro, validacion de campos unicos y perfil.

```
CODIGO  
builder.Services.AddScoped<AuthService>();
```

Servicios de autenticacion - Login con cookies y verificacion de credenciales.

```
CODIGO  
builder.Services.AddScoped<AeropuertoService>();
```

Servicios de vuelos - Busqueda por codigo IATA y listado de aeropuertos.

```
CODIGO  
builder.Services.AddScoped<VueloService>();
```

Servicios de vuelos - Busqueda con escalas, disponibilidad y precios.

```
CODIGO  
builder.Services.AddScoped<ReservacionService>();
```

Servicios de reservaciones - Creacion, consulta por usuario y cancelacion.

```
CODIGO  
builder.Services.AddScoped<ComentarioService>();
```

Servicios sociales - Agregar, listar y validar resenas de vuelos.

```
CODIGO  
builder.Services.AddScoped<DownService>();
```

Servicios sociales - Agregar, actualizar y eliminar votos en comentarios.

```
CODIGO  
builder.Services.AddScoped<AvionService>();
```

Servicios de flota - Gestion de aviones y capacidad.

```
CODIGO  
builder.Services.AddScoped<TripulacionService>();
```

Servicios de flota - Asignacion y consulta de tripulacion por vuelo.

```
CODIGO  
builder.Services.AddScoped<GestionReservacionService>();
```

Servicios administrativos - Gestion de reservaciones para el panel admin.

```
CODIGO  
builder.Services.AddScoped<AdminVueloService>();
```

Servicios administrativos - Crear, editar y cancelar vuelos.

```
CODIGO  
builder.Services.AddScoped<PerfilService>();
```

Servicios de usuarios - Consulta y actualizacion de datos personales del perfil.

```
CODIGO  
builder.Services.AddScoped<PdfService>();
```

Servicios de pagos - Generacion de facturas. PDF deshabilitado temporalmente mientras la libreria nativa wkhtmltopdf no este disponible en el entorno.

```
CODIGO  
builder.Services.AddScoped<FacturaService>();
```

Servicios de pagos - Generacion y consulta de facturas por reservacion.

```
CODIGO  
builder.Services.AddScoped<MetricasService>();
```

Servicios administrativos - Estadisticas del sistema para el dashboard.

```
CODIGO  
builder.Services.AddScoped<RutaService>();
```

Servicios de vuelos - Consulta y administracion de rutas aereas.

```
CODIGO  
builder.Services.AddScoped<AsientoService>();
```

Servicios de vuelos - Seleccion y disponibilidad de asientos por vuelo.

```
CODIGO  
builder.Services.AddScoped<AgenciaService>();
```

Servicios de agencias - Registro, handshake y gestion de estado de agencias.

```
CODIGO  
builder.Services.AddScoped<HandshakeService>();
```

Servicios de agencias - Intercambio de tokens con agencias externas (handshake).

```
CODIGO  
builder.Services.AddScoped<AgenciaAuthMiddleware>();
```

Servicios de agencias - Validacion de tokens de agencias en cada request.

```
CODIGO  
builder.Services.AddScoped<RutaAgenciaService>();
```

Servicios de agencias - Rutas disponibles para agencias via REST.


```
CODIGO  
builder.Services.AddScoped<VueloAgenciaService>();
```

Servicios de agencias - Búsqueda de vuelos para agencias via REST.

```
CODIGO  
builder.Services.AddScoped<ReservacionAgenciaService>();
```

Servicios de agencias - Crear y consultar reservaciones via REST.

```
CODIGO  
builder.Services.AddScoped<AsientoAgenciaService>();
```

Servicios de agencias - Disponibilidad de asientos via REST.

```
CODIGO  
builder.Services.AddScoped<ConfirmarReservacionAgenciaService>();
```

Servicios de agencias - Confirmar reservaciones creadas por agencias externas.

```
CODIGO  
builder.Services.AddScoped<HotelAliadoService>();
```

Servicios de Hotel Aliado - Consulta y agregacion de resultados de hoteles aliados.

```
CODIGO  
builder.Services.AddHttpClient();
```

IHttpClientFactory - Habilita llamadas HTTP salientes hacia sistemas externos (hotel aliado). Requerido por HandshakeHotelService para el proceso de handshake.

```
CODIGO  
builder.Services.AddScoped<HandshakeHotelService>();
```

Servicios de Hotel Aliado - Inicia el handshake de autenticacion con un hotel aliado y guarda el token de sesion resultante en HotelAliado.TokenHASH.

```
CODIGO  
builder.Services.AddSingleton<BusquedaTemporalService>();
```

Almacena búsquedas temporales en memoria - no persiste en base de datos.

```
CODIGO  
builder.Services.AddHostedService<ReservasCleanupService>();
```

Background service que limpia reservaciones que excedieron el tiempo de pago.

```
CODIGO  
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
```

Autenticacion por cookies - sesion de 8 horas con sliding expiration. Retorna 401 si no autenticado y 403 si no tiene permisos.

```
CODIGO  
builder.Services.AddAuthorization();
```

Habilita autorizacion basada en roles: admin, usuario y webservice.

```
CODIGO  
builder.Services.AddCors(options =>
```

CORS para permitir peticiones del frontend Svelte. Acepta origenes configurados en appsettings.json o los puertos por defecto de Vite.

```
CODIGO  
var corsOrigins = builder.Configuration
```

```
CODIGO  
var app = builder.Build();
```

Construye la aplicacion con toda la configuracion registrada.

```
CODIGO  
app.UseCors();
```

Pipeline de middleware - el orden importa. Primero CORS para que las preflight requests pasen.

```
CODIGO  
app.UseAuthentication();
```

Autenticacion: lee la cookie de sesion y valida la identidad del usuario.

```
CODIGO  
app.MapHealthChecks("/health");
```

Health check endpoint - GET /health retorna 200 OK si el servidor esta activo.

```
CODIGO  
app.UseAuthorization();
```

Autorizacion: valida roles despues de confirmar la identidad.

```
CODIGO  
app.MapControllers();
```

Mapea todos los controllers de la API descubiertos automaticamente.